

A Self-Explaining Temporal Graph Learning Method for Low-Latency Financial Fraud Detection and Decision Support

Abstract

Fraud screening systems must rank suspicious transactions quickly and tell the analyst why a transaction was flagged. Graph neural networks (GNNs) exploit transaction relations but usually rely on post-hoc explainers, which are costly and may not reflect the model. We propose RTXGNN, a self-explaining temporal GNN whose learned feature, edge and node masks gate message passing and are returned as the explanation of every score. A training objective makes the top- k explanation sufficient for the prediction and its complement uninformative, and a shift-invariant temporal encoding (HRAPE) supplies time gaps, recency and transaction velocity. On the Elliptic Bitcoin dataset with a temporal split and 16 baselines over 10 seeds, RTXGNN reaches a test F1 of 0.700 and an average precision of 0.771, on par with the strongest GNN baselines and higher on average than the self-explaining SEFraud (+0.054 F1). Gradient-boosted trees on the same engineered features remain the most accurate model (F1 0.790). Its ranking reaches a precision of 0.99 among the 500 highest-scored test transactions. Its explanations are nearly as sufficient as Integrated Gradients at about one hundredth of the cost, and far more stable than GNNExplainer. On a synthetic benchmark with planted laundering rings, the most important incoming transaction of a detected ring member is a ring transaction in 91% of the cases. Scoring and explaining a transaction end to end takes 5.9 ms at the 99th percentile on one CPU core. We also analyse the temporal encoding, drift and retraining, label efficiency and a second financial dataset.

Keywords: Financial fraud analytics, Explainable analytics, Graph neural networks, Decision support, Temporal networks, Anti-money laundering

List of abbreviations

Table 1: Abbreviations used in this paper.

Abbreviation	Full term
<i>Regulatory and legal</i>	
AML/CFT	Anti-Money Laundering / Countering the Financing of Terrorism
ECOA	Equal Credit Opportunity Act
FATF	Financial Action Task Force
GDPR	General Data Protection Regulation
MiCA	Markets in Crypto-Assets Regulation
<i>Machine learning</i>	
AP	Average Precision (area under the precision–recall curve)
AUC	Area Under the Receiver Operating Characteristic Curve
Fid ⁺ / Fid [−]	Fidelity when the explanation is removed / kept alone
GCN, GAT	Graph Convolutional Network, Graph Attention Network
GNN	Graph Neural Network
GRU, LSTM	Gated Recurrent Unit, Long Short-Term Memory
IG	Integrated Gradients
KL	Kullback–Leibler divergence
LR, RF	Logistic Regression, Random Forest
MLP	Multi-Layer Perceptron
XAI	Explainable Artificial Intelligence
<i>Proposed method</i>	
RTXGNN	Real-Time eXplainable Graph Neural Network (the proposed method)
HRAPE	Hierarchical Recency-Aware Positional Encoding
SEAL	Self-Explainable Aggregation Layer
<i>Baselines</i>	
APAN	Asynchronous Propagation Attention Network
CARE-GNN	Camouflage-Resistant Graph Neural Network
EvolveGCN	Evolving Graph Convolutional Network
FRAUDRE	Fraud detection dual-resistant to graph inconsistency and imbalance

Continued on next page

Abbreviation	Full term
GAS	GCN-based Anti-Spam model
PC-GNN	Pick-and-Choose Graph Neural Network
SEFraud	Self-Explainable Fraud detection
TGAT, TGN	Temporal Graph Attention network, Temporal Graph Network

1. Introduction

Illicit activity in cryptocurrency markets was estimated at USD 24.2 billion in 2023¹. Blockchain transactions are pseudonymous and irreversible, and funds can be moved through long chains of intermediate addresses. Regulators have extended anti-money-laundering obligations to virtual asset service providers, for example through the FATF guidance on virtual assets and the EU Markets in Crypto-Assets (MiCA) regulation²³. Institutions are expected to flag suspicious transactions promptly and to document why a transaction was flagged.

Graph Neural Networks (GNNs) are a natural model for this setting because they use the relations between transactions and accounts (Kipf and Welling, 2017; Veličković et al., 2018; Hamilton et al., 2017). Fraud-specific architectures such as CARE-GNN (Dou et al., 2020) and PC-GNN (Liu et al., 2021) address camouflage and class imbalance. Two practical obstacles remain for decision support. First, a flagged transaction is reviewed by an analyst, who needs to know which inputs drove the score. Post-hoc explainers such as GNNExplainer (Ying et al., 2019) and PGExplainer (Luo et al., 2020) provide this, but they require an additional optimisation or model per explanation and are not guaranteed to reflect the model’s reasoning (Faber et al., 2021). Second, transaction networks change over time, and a model trained on past data can fail abruptly when the underlying activity changes (Weber et al., 2019; Gama et al., 2014). In addition, strong feature-based

¹<https://go.chainalysis.com/crypto-crime-2024.html>

²<https://www.fatf-gafi.org/en/publications/Fatfrecommendations/Guidance-rba-virtual-assets-2021.html>

³<https://www.esma.europa.eu/esmas-activities/digital-finance-and-innovation/markets-crypto-assets-regulation-mica>

learners such as random forests are hard to beat on engineered transaction features (Weber et al., 2019; Grinsztajn et al., 2022). Claims of an advantage for a graph model therefore require careful comparison.

Self-explaining GNNs compute an explanation in the same forward pass as the prediction. SEFraud (Li et al., 2024) learns feature and edge masks for fraud detection on static heterogeneous graphs and reports industrial deployment. Temporal GNNs such as EvolveGCN (Pareja et al., 2020), TGAT (Xu et al., 2020), TGN (Rossi et al., 2020) and APAN (Wang et al., 2021) model the evolution of graphs, but do not produce explanations.

Objective and approach

This paper studies whether a self-explaining temporal GNN can provide explanations that are faithful to its predictions, at a latency compatible with online screening, while remaining competitive in detection accuracy with strong graph and non-graph baselines. We propose **RTXGNN**, which combines two components:

1. **Hierarchical Recency-Aware Positional Encoding (HRAPE)**, a temporal encoding built only from relative quantities: multi-scale sinusoidal codes of the time gap between a transaction and the scored node, a learnable recency bias of the attention, and transaction counts in trailing windows (velocity). Because HRAPE never encodes the absolute timestamp, it does not extrapolate to unseen time values when the model scores future periods.
2. **Self-Explainable Aggregation Layers (SEAL)**, attention-based message-passing layers whose messages are gated by learned edge and node masks, preceded by a per-node feature mask on the raw inputs. The masks that gate the computation are returned as the explanation.

The two are trained with an objective that makes the reported explanation faithful to the prediction. The top- k explanation alone must reproduce the prediction (sufficiency), and the inputs outside the explanation must not carry information about the label (necessity). A sparsity term keeps explanations short.

We evaluate RTXGNN on the Elliptic Bitcoin dataset (Weber et al., 2019) with a temporal split (train on time steps 1–30, validate on 31–34, test on 35–49), on the T-Finance transaction network (Tang et al., 2022), and on a synthetic benchmark in which laundering rings differ from decoy cycles

only in the timing of their transactions. We compare with 16 baselines: feature-only models (logistic regression, random forest, XGBoost, MLP), general GNNs, temporal GNNs (EvolveGCN, TGAT, TGN, APAN) and fraud-specific GNNs (CARE-GNN, PC-GNN, GAS, FRAUDRE, SEFraud). All results are averaged over 10 seeds with paired significance tests.

Summary of findings

On Elliptic, RTXGNN reaches a test F1 of 0.700 and an AP of 0.771. This is on par with the best graph neural network baseline, GAS (F1 0.701), and the differences to most other GNNs are within the variation across seeds. On average RTXGNN is ahead of the self-explaining SEFraud (+0.054 F1, +0.029 AP), although neither difference remains significant after correction for multiple comparisons. Gradient-boosted trees on the same engineered features remain the most accurate model (XGBoost, F1 0.790), and the random forest reaches 0.733. For decision support the ranking is what matters: the 500 highest-scored test transactions have a precision of 0.99 (Section 5.3). Reading the explanation from the masks adds 0.24 ms to a single-transaction request. The explanations’ sufficiency is close to that of Integrated Gradients, which is about one hundred times more expensive, and they are far more stable than GNNExplainer (Section 5.8). On a synthetic benchmark with planted laundering rings, the most important transaction identified by RTXGNN is a ring transaction for 91% of the detected ring members (Section 5.9). All models fail after the dark-market shutdown in the Elliptic test period, and retraining with a one-step label delay recovers part of the loss (Section 5.6). End-to-end scoring and explanation of a single transaction, including neighbourhood sampling, takes 5.9 ms at the 99th percentile on one CPU core (Section 5.11).

Contributions

1. **A self-explaining temporal GNN with a faithfulness objective.** RTXGNN gates message passing with learned feature, edge and node masks, and it is trained with explicit sufficiency and necessity terms. These terms rule out masks that are not used by the model. HRAPE uses only shift-invariant temporal information. In our ablation it is more accurate than Time2Vec on the same time gaps, and adding the absolute timestamp makes performance less stable across seeds under a temporal split.

2. **A reproducible, statistically controlled comparison.** We compare 16 baselines, including tree ensembles and temporal GNNs, under one protocol with 10 seeds and Holm-corrected paired tests, and we release all code. An equal-budget hyperparameter search for RTXGNN and ten baselines, selected on validation data only, leaves the conclusions unchanged. The evaluation includes ablations that replace HRAPE with Time2Vec, a comparison of mask sparsity with dropout and weight decay, stratified label-efficiency experiments with repeated subsets, a per-time-step drift analysis with a retraining strategy, and measured end-to-end CPU latency with tail percentiles.
3. **An evaluation of explanations for decision support.** We measure the sufficiency and necessity of feature explanations at several sizes against saliency, Integrated Gradients, GNNExplainer, SEFraud’s masks and random explanations, together with class-wise results, stability and cost. We test edge explanations against known ground truth on synthetic data, translate detection scores into alert-queue terms, and discuss what is needed to use such explanations in regulated review processes.

Section 2 reviews related work. Section 3 presents RTXGNN. Section 4 describes the datasets, baselines and protocol, and Section 5 the results. Section 6 discusses decision support, regulation, deployment and limitations, and Section 7 concludes.

2. Related work

We review graph neural networks for fraud detection, explainability methods for GNNs and how they are evaluated, temporal graph learning, and low-latency deployment. We then position RTXGNN against the closest methods.

2.1. Graph neural networks for fraud detection

Graph Neural Networks model the relational structure of transaction networks. Graph Convolutional Networks (GCN) (Kipf and Welling, 2017; Asiri and Somasundaram, 2025), Graph Attention Networks (GAT) (Veličković et al., 2018; Fu et al., 2026) and GraphSAGE (Hamilton et al., 2017; Tang, 2025) established the message-passing framework on which most fraud detectors build. GCN aggregates neighbours with a normalised adjacency, GAT

learns attention weights over neighbours, and GraphSAGE learns inductive aggregators that generalise to unseen nodes, which matters when new accounts or transactions arrive continuously.

Fraud graphs violate several assumptions of these architectures. **Camouflage**, in which fraudsters mimic legitimate users, weakens neighbourhood aggregation. CARE-GNN (Dou et al., 2020) selects neighbours with a label-aware similarity measure and a reinforcement-learning threshold. **Class imbalance** is addressed by PC-GNN (Liu et al., 2021) with label-balanced sampling of training nodes and neighbour selection. FRAUDRE (Zhang et al., 2021) tackles both graph inconsistency and imbalance with a fraud-aware convolution that uses node-neighbour differences, a combination of intermediate layer outputs, and an imbalance-oriented classifier. GAS (Li et al., 2019), deployed at Xianyu, combines aggregation over the interaction graph with aggregation over a similarity graph built from content features. Liu et al. (2020) characterise context, feature and relation inconsistencies and propose GraphConsis. H2-FDetector (Shi et al., 2022) separates homophilic from heterophilic edges, GeniePath (Liu et al., 2019) learns adaptive receptive fields, and DGA-GNN (Duan et al., 2024) groups non-additive attributes with decision-tree binning. Tang et al. (2022) relate fraud and anomaly detection on graphs to spectral “right-shift” effects and release the T-Finance dataset used in our study. GADBench (Tang et al., 2023) shows that tree ensembles with neighbourhood aggregation are strong competitors to specialised GNNs on many anomaly-detection benchmarks.

On the Elliptic Bitcoin data specifically, Weber et al. (2019) found that a random forest on the transaction features outperforms GCN, and that all models fail after a dark-market shutdown in the test period. This is consistent with the broader observation that tree ensembles remain very strong on tabular features (Grinsztajn et al., 2022). We therefore include logistic regression, random forest (Breiman, 2001) and XGBoost (Chen and Guestrin, 2016) as baselines, in addition to general, temporal and fraud-specific GNNs.

2.2. Explainability methods for graph neural networks

Post-hoc explainers explain a trained model. GNNExplainer (Ying et al., 2019) optimises soft masks over edges and features that preserve a prediction and must be run separately for every instance. PGExplainer (Luo et al., 2020) amortises this optimisation with a parametric edge-mask network. SubgraphX (Yuan et al., 2021) searches subgraphs with Monte Carlo tree search and Shapley values and is accurate but slow. CF-GNNExplainer

(Lucic et al., 2022) finds minimal edge deletions that change a prediction, and CF² (Tan et al., 2022) combines factual (sufficient) and counterfactual (necessary) criteria. GraphLIME (Huang et al., 2023) fits local surrogate models, XGNN (Yuan et al., 2020) gives model-level explanations, and gradient-based attributions such as saliency (Simonyan et al., 2014) and Integrated Gradients (Sundararajan et al., 2017) apply to GNNs directly (Pope et al., 2019; Baldassarre and Azizpour, 2019). Yuan et al. (2023) survey the field.

Self-explaining GNNs compute explanations during the forward pass. GSAT (Miao et al., 2022) learns stochastic attention under an information bottleneck. SEFraud (Li et al., 2024) learns feature and edge masks for fraud detection on heterogeneous graphs with a triplet loss that separates the masked from the complementary representation, and reports deployment at the Industrial and Commercial Bank of China. SEFraud and GSAT operate on static graphs. RTXGNN adopts the self-explaining design but uses an objective that ties the reported top- k explanation to the prediction through explicit sufficiency and necessity terms, and it adds relative temporal encodings. We use the term “self-explaining” rather than “interpretable”: a network with several non-linear layers remains a complex model even when it exposes attributions (Rudin, 2019; Lipton, 2018).

Evaluating explanations is difficult. Faber et al. (2021) show that comparing explanations with synthetic ground truth can mislead, because a model may solve a task with a rationale different from the planted one. Fidelity metrics measure instead how the prediction changes when the explanation is removed (Fid⁺, necessity) or kept alone (Fid⁻, sufficiency) (Yuan et al., 2021; Amara et al., 2022). Agarwal et al. (2023) add stability and fairness criteria. We report Fid⁺ and Fid⁻ at several explanation sizes, with a random explanation as reference. We also report stability under input perturbation and agreement across independently trained models, class-wise results, and the cost of each explainer. Ground truth is used only on the synthetic benchmark, where the planted rings are known to be the only signal separating the classes.

2.3. Temporal graph neural networks

Discrete-time models operate on snapshots. EvolveGCN (Pareja et al., 2020) evolves the GCN weights with a recurrent network (EvolveGCN-O/-H), DySAT (Sankar et al., 2020) applies structural and temporal self-attention, and ROLAND (You et al., 2022) turns static GNNs into incremental dynamic models. Continuous-time models consume timestamped events. TGAT (Xu

et al., 2020) encodes time gaps with a functional time encoding inside temporal attention. TGN (Rossi et al., 2020) adds a per-node memory updated by a recurrent cell from event messages and shows that JODIE, DyRep and TGAT are special cases. APAN (Wang et al., 2021) decouples inference from graph queries with asynchronous mailboxes, a design aimed at real-time serving. Time2Vec (Kazemi et al., 2019) is a generic learnable time embedding with one linear and several periodic components.

Continuous-time models assume that nodes interact repeatedly over time. In Elliptic each transaction exists in one of 49 snapshots and all edges connect transactions of the same snapshot (Weber et al., 2019). The memory of TGN and the mailbox of APAN therefore carry no information across snapshots, and time gaps within a snapshot are zero. We nevertheless include TGAT, TGN, APAN and EvolveGCN as baselines, because they are the standard temporal GNNs, and describe their adaptation in Appendix B. DySAT’s temporal attention runs over the snapshots in which a node appears. Every Elliptic transaction appears in exactly one snapshot, so DySAT reduces to a structural attention network (GAT), which is already a baseline.

2.4. Low-latency fraud detection systems

Production fraud detection systems have end-to-end latency budgets of roughly 10–100 ms. BRIGHT (Lu et al., 2022), deployed at eBay, pre-computes entity embeddings in a batch layer and runs only a light real-time layer for new transactions. TitAnt (Cao et al., 2019; Dong and Xiao, 2025) and GEM (Liu et al., 2018; Rao et al., 2025) serve predictions at Ant Financial and Alipay. These systems return scores without explanations. When explanations are needed, they are usually produced asynchronously for the cases that are escalated. Latency figures reported in the literature depend strongly on hardware, batch size and whether neighbourhood retrieval is included. We therefore report our own end-to-end measurements on commodity CPU hardware, with every stage of the pipeline timed separately.

Beyond graph data, supervised learning is widely used for credit-card and insurance fraud. Afriyie et al. (2023) compare logistic regression, decision trees and random forests for credit-card fraud and find the random forest strongest under class imbalance. Jacob et al. (2025) combine optimisation-based feature selection with a BERT-LSTM model for insurance claims. These results reinforce the need to compare graph models against strong feature-based learners.

2.5. Regulation and explanation requirements

GDPR Article 22⁴ restricts solely automated decisions with legal or similarly significant effects, and Articles 13–15 require “meaningful information about the logic involved”. Whether this amounts to a right to an explanation is debated (Wachter et al., 2017; Selbst and Powles, 2017; Goodman and Flaxman, 2016; Sargeant, 2026). In U.S. consumer credit, the Equal Credit Opportunity Act (U.S. Congress, 1974) and Regulation B require specific principal reasons for adverse actions. The CFPB has stated that this obligation also applies when complex algorithms are used⁵. For virtual assets, FATF guidance⁶ and the EU MiCA regulation (Teng et al., 2026) require AML/CFT controls that are documented and auditable. Supervisory guidance on model risk management, such as the U.S. SR 11-7 (Board of Governors of the Federal Reserve System and Office of the Comptroller of the Currency, 2011), requires that models be validated and their limitations understood.

These texts set requirements for processes and for the content of notices, not for model architectures. A feature attribution can contribute to a reason statement or an audit record, but it does not by itself establish compliance with any of them. This applies especially when the features are anonymised, as in Elliptic. Section 6.2 discusses what would be needed to operationalise RTXGNN’s explanations. Cherif et al. (2024) note that GNN explanations have not yet been validated with analysts or regulators, and our work does not close this gap.

2.6. Positioning of RTXGNN

Table 2 compares RTXGNN with the closest methods on properties that can be verified from the methods themselves. We do not include a “regulatory compliance” property, because compliance is a property of an institution’s process and cannot be verified from a method, and we report “explanation in the forward pass” rather than “real-time explanation”, since latency depends on the deployment.

⁴<https://eur-lex.europa.eu/eli/reg/2016/679/oj>

⁵<https://www.consumerfinance.gov/compliance/circulars/circular-2022-03-adverse-action-notification-requirements-in-connection-with-credit-decisions-based-on-complex-algorithms/>

⁶<https://www.fatf-gafi.org/en/publications/Fatfrecommendations/Guidance-rba-virtual-assets-2021.html>

Table 2: Properties of related methods. **F** = supported, *P* = partially supported, $-$ = not supported. Temporal: uses timestamps or snapshots. Self-expl.: explanation computed in the forward pass. Fraud: design for camouflage or imbalance. Feat./Edge: explanation over input features / edges. Obj.: explanation tied to the prediction by a sufficiency/necessity objective.

Method	Temporal	Self-expl.	Fraud	Feat.	Edge	Obj.
GCN / GraphSAGE (Kipf and Welling, 2017; Hamilton et al., 2017)	$-$	$-$	$-$	$-$	$-$	$-$
GAT (Velićković et al., 2018)	$-$	<i>P</i>	$-$	$-$	<i>P</i>	$-$
CARE-GNN / PC-GNN (Dou et al., 2020; Liu et al., 2021)	$-$	$-$	F	$-$	$-$	$-$
FRAUDRE / GAS (Zhang et al., 2021; Li et al., 2019)	$-$	$-$	F	$-$	$-$	$-$
EvolveGCN (Pareja et al., 2020)	F	$-$	$-$	$-$	$-$	$-$
TGAT / TGN / APAN (Xu et al., 2020; Rossi et al., 2020; Wang et al., 2021)	F	<i>P</i>	$-$	$-$	<i>P</i>	$-$
GNNE explainer (Ying et al., 2019) (post hoc)	$-$	$-$	$-$	F	F	<i>P</i>
PGExplainer (Luo et al., 2020) (post hoc)	$-$	$-$	$-$	$-$	F	<i>P</i>
GSAT (Miao et al., 2022)	$-$	F	$-$	$-$	F	<i>P</i>
SEFraud (Li et al., 2024)	$-$	F	F	F	F	<i>P</i>
RTXGNN (ours)	<i>P</i>	F	<i>P</i>	F	F	F

RTXGNN’s temporal support is marked partial because it uses relative time only, which is uninformative on Elliptic (Section 3.2). Its fraud-specific support is partial because it relies on class-balanced training rather than on a camouflage-specific mechanism. Attention weights (GAT, TGAT) are marked partial because attention is not necessarily faithful to the prediction.

3. Methodology

This section formalises the problem (Section 3.1) and describes the two components of RTXGNN: the Hierarchical Recency-Aware Positional Encoding (HRAPE, Section 3.2) and the Self-Explainable Aggregation Layer (SEAL, Section 3.3). Section 3.4 presents the training objective and Section 3.5 the inference procedure used for low-latency scoring. Figure 1 gives an overview. Every equation in this section corresponds one-to-one to the released implementation (module `rtxgnn/model.py` for the architecture and `rtxgnn/objective.py` for the objective).

Terminology. Following Rudin (2019) and Lipton (2018), we distinguish *interpretable* models, whose decision logic can be inspected directly (e.g. sparse linear models, shallow decision trees), from *self-explaining* models, which remain complex non-linear predictors but compute an explanation as part of the forward pass. RTXGNN belongs to the second category. Its masks are attributions: they state which inputs the prediction depended on, not the full decision logic. We therefore do not describe it as intrinsically interpretable.

3.1. Problem formulation

A transaction network is a directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. Each node $v \in \mathcal{V}$ (a transaction in Elliptic, an account in T-Finance and in the synthetic

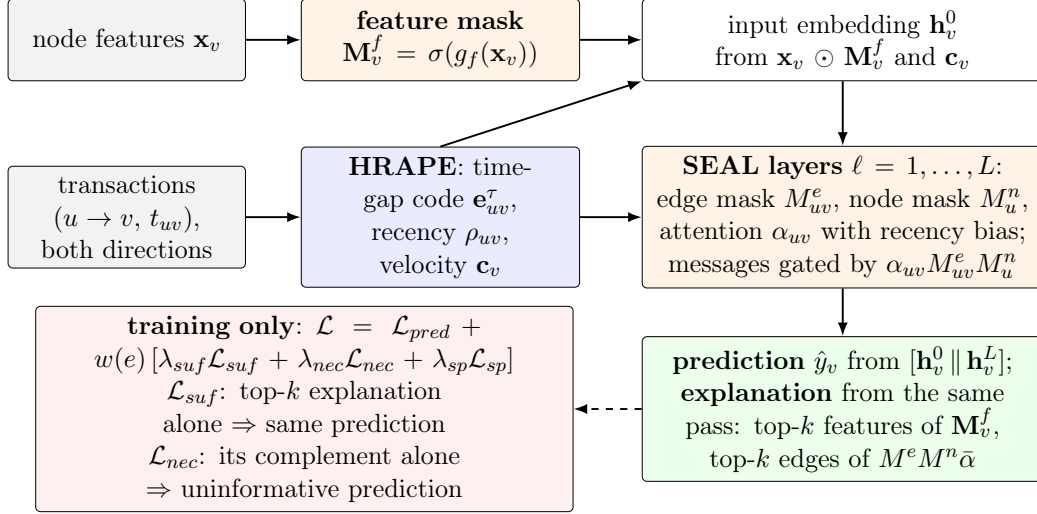


Figure 1: Overview of RTXGNN. The masks that form the explanation are the gates that produce the prediction, so both come out of one forward pass. The objective in the bottom box is used only during training. It evaluates the model twice more, once on the top- k explanation and once on its complement (Section 3.4).

benchmark) has a feature vector $\mathbf{x}_v \in \mathbb{R}^d$ and a timestamp τ_v , the time at which it has to be scored. Each edge $(u \rightarrow v) \in \mathcal{E}$ is a flow of funds with timestamp $t_{uv} \leq \tau_v$. Message passing uses both directions of every edge. The reversed copy $(v \rightarrow u)$ carries the type $r = 1$ and the original the type $r = 0$, so the model can distinguish incoming from outgoing funds. The time gap of a directed edge is $\Delta_{uv} = |\tau_v - t_{uv}|$.

Given the labelled nodes observed up to a cut-off time, the task is to estimate for every later node the probability $\hat{y}_v = f_\theta(\mathcal{G}, v) \in [0, 1]$ that it is illicit. At the same time the model returns an explanation $\mathcal{X}_v$: a ranking of the node’s own features and a ranking of its incoming edges by their contribution to $\hat{y}_v$. The decision threshold on $\hat{y}_v$ is chosen on validation data (Section 4.4).

3.2. Hierarchical Recency-Aware Positional Encoding (HRAPE)

HRAPE supplies three kinds of temporal information to SEAL. It depends only on time *differences* and on counts in trailing windows, never on the absolute timestamp, so a model evaluated on future time steps never sees an out-of-range time value. In our ablation (Section 5.4), additionally encoding

the absolute timestamp lowers the mean test F1 and makes it markedly less stable across seeds.

Multi-scale gap encoding. For a set of periods $P_1, \dots, P_S$ (in the time unit of the data) the gap of an edge is encoded as

$$\phi(\Delta) = [\sin(2\pi\Delta/P_s), \cos(2\pi\Delta/P_s)]_{s=1}^S, \quad \mathbf{e}_{uv}^\tau = \mathbf{W}_\Delta \phi(\Delta_{uv}), \quad (1)$$

with $\mathbf{W}_\Delta \in \mathbb{R}^{D \times 2S}$ learnable. We use $P = (2, 4, 13, 26)$ time steps for Elliptic (one month to one year) and $P = (1, 7, 30, 90)$ days for the synthetic benchmark.

Recency. A learnable decay turns the gap into a recency score that biases attention (Eq. 8):

$$\rho_{uv} = \exp(-\text{softplus}(\lambda) \Delta_{uv}). \quad (2)$$

Velocity. Burst behaviour is captured by the number of transactions a node received in trailing windows $w_1 < w_2 < w_3$ before τ_v ,

$$\mathbf{c}_v = \left[\log(1 + |\{(u \rightarrow v) : 0 \leq \tau_v - t_{uv} < w_k\}|) \right]_{k=1}^3, \quad (3)$$

with $w = (1, 2, 4)$ steps for Elliptic and $(1, 7, 30)$ days for the synthetic data. The velocity vector enters the input embedding (Eq. 5).

What HRAPE can and cannot do on Elliptic. In Elliptic every edge connects two transactions of the same time step and each transaction appears in exactly one time step (Weber et al., 2019). Hence $\Delta_{uv} = 0$ for all edges: the gap code is a constant, $\rho_{uv} = 1$, and the velocity vector reduces to the node’s in-degree within its time step. On Elliptic HRAPE therefore contributes little beyond a degree feature, and our ablation (Section 5.4) confirms this. The temporal mechanisms become informative only on data with continuous timestamps. We test this on the synthetic benchmark (Section 5.9), where laundering rings differ from decoy cycles only in the timing of their hops. Hour-of-day or day-of-week encodings cannot be computed from the public Elliptic data and are therefore not used.

3.3. Self-Explainable Aggregation Layer (SEAL)

Input feature mask and embedding. A two-layer MLP $g_f : \mathbb{R}^d \rightarrow \mathbb{R}^d$ produces a per-node, per-feature mask that gates the raw input features:

$$\mathbf{M}_v^f = \sigma(g_f(\mathbf{x}_v) + 2) \in (0, 1)^d, \quad (4)$$

$$\mathbf{h}_v^0 = \text{ReLU}(\mathbf{W}_{in}(\mathbf{x}_v \odot \mathbf{M}_v^f) + \mathbf{W}_c \mathbf{c}_v). \quad (5)$$

The constant $+2$ initialises the mask close to one, so that training starts from the unmasked model. The feature mask is applied to the raw input features, where it is an attribution over identifiable inputs, and not to hidden representations.

Masks of layer ℓ . Given $\mathbf{h}^{\ell-1}$, each SEAL layer computes a node (source) mask and an edge mask,

$$M_u^n = \sigma(g_n(\mathbf{h}_u^{\ell-1})), \quad (6)$$

$$M_{uv}^e = \sigma(\mathbf{w}^\top \text{ReLU}(\mathbf{A}\mathbf{h}_v^{\ell-1} + \mathbf{B}\mathbf{h}_u^{\ell-1} + \mathbf{C}_{r_{uv}} + \mathbf{T}\mathbf{e}_{uv}^\tau) + b), \quad (7)$$

where g_n is a two-layer MLP and $\mathbf{A}, \mathbf{B}, \mathbf{T} \in \mathbb{R}^{D \times D}$, $\mathbf{C}_r \in \mathbb{R}^D$. Eq. (7) is an MLP applied to the concatenation $[\mathbf{h}_v \parallel \mathbf{h}_u \parallel \mathbf{C}_r \parallel \mathbf{e}_{uv}^\tau]$, written in a factorised form that is evaluated with node-level matrix products.

Attention with recency bias. With H heads of size $D_k = D/H$,

$$\begin{aligned} \mathbf{q}_v &= \mathbf{W}_Q \mathbf{h}_v^{\ell-1}, & \mathbf{k}_{uv} &= \mathbf{W}_K \mathbf{h}_u^{\ell-1} + \mathbf{E}_{r_{uv}}^K + \mathbf{e}_{uv}^\tau, \\ \mathbf{m}_{uv} &= \mathbf{W}_V \mathbf{h}_u^{\ell-1} + \mathbf{E}_{r_{uv}}^V + \mathbf{e}_{uv}^\tau, \\ \alpha_{uv}^{(i)} &= \text{softmax}_{u \in \mathcal{N}(v)} \left(\mathbf{q}_v^{(i)\top} \mathbf{k}_{uv}^{(i)} / \sqrt{D_k} + \gamma \rho_{uv} \right), & i &= 1, \dots, H, \end{aligned} \quad (8)$$

where $^{(i)}$ selects the coordinates of head i and γ is learnable.

Gated aggregation and update. The masks gate the attention-weighted messages *after* normalisation, so an edge with $M_{uv}^e \rightarrow 0$ is removed from the computation instead of being compensated by a renormalised softmax:

$$\mathbf{a}_v = \mathbf{W}_O \left\|_{i=1}^H \sum_{u \in \mathcal{N}(v)} \alpha_{uv}^{(i)} M_{uv}^e M_u^n \mathbf{m}_{uv}^{(i)}, \quad (9)$$

$$\tilde{\mathbf{h}}_v = \text{LN}(\mathbf{h}_v^{\ell-1} + \text{Dropout}(\mathbf{a}_v)), \quad \mathbf{h}_v^\ell = \text{LN}(\tilde{\mathbf{h}}_v + \text{FFN}(\tilde{\mathbf{h}}_v)). \quad (10)$$

Prediction and explanation. After L layers the prediction uses both the input embedding and the propagated representation (a skip connection that matters on Elliptic, where the node’s own features are highly informative):

$$\hat{y}_v = \sigma(\text{MLP}([\mathbf{h}_v^0 \parallel \mathbf{h}_v^L])). \quad (11)$$

The explanation of v is read from the same forward pass:

$$\mathcal{X}_v = \left(\text{TopK}_{k_f}(\mathbf{M}_v^f), \text{TopK}_{k_e} \{I_{uv} = M_{uv}^e M_u^n \bar{\alpha}_{uv} : u \in \mathcal{N}(v)\} \right), \quad (12)$$

with $\bar{\alpha}_{uv}$ the head-averaged attention of the last layer. The first part ranks the node’s own features, the second ranks its incoming (and, through the reversed copies, outgoing) transactions. Because the edge importance I_{uv} is the gate that multiplied the message of u in Eq. (9), an edge with a small I_{uv} had a proportionally small influence on $\mathbf{h}_v^L$.

3.4. Training objective

Prediction loss. We use a class-balanced binary cross-entropy in which each class contributes half of the loss,

$$\mathcal{L}_{pred} = \frac{1}{|\mathcal{V}_{tr}|} \sum_{v \in \mathcal{V}_{tr}} \left[\frac{y_v}{2\pi} \ell(\hat{y}_v, 1) + \frac{1-y_v}{2(1-\pi)} \ell(\hat{y}_v, 0) \right], \quad (13)$$

where ℓ is the log-loss and π the fraction of illicit labels among the training nodes $\mathcal{V}_{tr}$. Focal loss (Lin et al., 2017) is evaluated as an ablation. Its values are two orders of magnitude smaller than those of the explanation terms below, which makes the weighting of the objective fragile.

Explanation views. Let $\text{top}_k(\cdot)$ be the indicator of the k largest entries: the k_f largest features of $\mathbf{M}_v^f$, and for every node and layer its k_e incoming edges with the largest M_{uv}^e . From the soft masks we build two hard views, using the straight-through estimator $\mathbf{M} + (\text{top}_k(\mathbf{M}) - \mathbf{M})_{\text{no-grad}}$ so that gradients reach the mask generators:

- the *explanation view* S keeps only the selected features and edges (all other gates are set to zero);
- the *complement view* C keeps only the non-selected features and edges.

Running the model on the two views gives $\hat{y}_v^S$ and $\hat{y}_v^C$ (M^n is kept soft in both views).

Sufficiency and necessity. With $\text{KL}(p\|q)$ the Kullback–Leibler divergence between Bernoulli distributions,

$$\mathcal{L}_{suf} = \frac{1}{|\mathcal{V}_{tr}|} \sum_{v \in \mathcal{V}_{tr}} \text{KL}(\text{sg}(\hat{y}_v) \parallel \hat{y}_v^S), \quad (14)$$

$$\mathcal{L}_{nec} = \frac{1}{|\mathcal{V}_{tr}|} \sum_{v \in \mathcal{V}_{tr}} \text{KL}(\pi \parallel \hat{y}_v^C), \quad (15)$$

where sg stops the gradient. $\mathcal{L}_{suf}$ asks that the top- k explanation alone reproduce the model’s prediction. $\mathcal{L}_{nec}$ asks that the remaining inputs alone carry no information about the label, i.e. that the prediction on the complement fall back to the class prior. These are the factual and counterfactual criteria of Tan et al. (2022), and they match the Fid^- and Fid^+ metrics used in evaluation (Yuan et al., 2021; Amara et al., 2022).

Why the objective cannot be met by ignoring the masks. A fidelity loss of the form $\text{KL}(f(\mathcal{G})\|f(\mathcal{G}_{masked}))$ with soft masks alone is minimised trivially by masks that are all close to one, or by a model that is insensitive to the masks, and says nothing about whether the masks are faithful. Our objective rules out both degenerate solutions: (i) the views are *hard*: in S all but k_f of the d features and all but k_e incoming edges are removed, so $\mathcal{L}_{suf}$ can only be small if the selected elements actually carry the prediction; (ii) a model that ignores the masks predicts the same value on the complement C as on the full input, which incurs $\mathcal{L}_{nec} > 0$ for every node whose prediction differs from the prior; (iii) a mask that is uninformative about importance (for instance uniform) produces a random top- k selection, for which both terms are large unless the model does not depend on the inputs at all, and that case is excluded by $\mathcal{L}_{pred}$. The objective uses only the model’s own predictions and the class prior, never the labels of the explained nodes, so it does not supervise the explanation with labels.

Sparsity. $\mathcal{L}_{sp}$ is the mean L_1 norm of all masks,

$$\mathcal{L}_{sp} = \overline{\mathbf{M}^f} + \frac{1}{L} \sum_{\ell=1}^L (\overline{M^{e,(\ell)}} + \overline{M^{n,(\ell)}}), \quad (16)$$

where the bar denotes the mean over all entries. This is ordinary L_1 regularisation of auxiliary gate activations. We do not claim it is a new form of regularisation, and Section 5.5 compares it with dropout and weight decay.

No label supervision of the gates. We deliberately do not supervise any mask or temporal gate with labels (for example by rewarding high gates on edges incident to fraudulent nodes). Such a term would leak label information into what is presented as an explanation.

Total objective and curriculum.

$$\mathcal{L} = \mathcal{L}_{pred} + w(e)(\lambda_{suf}\mathcal{L}_{suf} + \lambda_{nec}\mathcal{L}_{nec} + \lambda_{sp}\mathcal{L}_{sp}), \quad w(e) = \min(1, e/e_w), \quad (17)$$

with epoch e , warm-up $e_w = 20$, $\lambda_{suf} = \lambda_{nec} = 0.5$, $\lambda_{sp} = 0.05$, $k_f = 10$ and $k_e = 2$. These values were fixed before the experiments reported here and were not tuned on test data.

3.5. Training and inference procedures

Algorithm 1 gives the training procedure. We train full-batch on the subgraph induced by the 2-hop neighbourhoods of the labelled training nodes. For a model with $L = 2$ message-passing layers this is exact: every training prediction depends only on the 2-hop neighbourhood of its node, so no neighbourhood sampling is used during training. Evaluation on validation and test nodes uses the 2-hop neighbourhoods of those nodes in the same way.

At inference time (Algorithm 2) a request consists of one or more transactions to score. For each target we sample at most $F = 25$ incoming neighbours per hop for two hops from an in-memory CSR adjacency, run one forward pass on the union of the sampled subgraphs, and read the explanation of every target from the masks of that pass (Eq. 12). On Elliptic the in-degree rarely exceeds F , so the sampled neighbourhood is almost always the exact 2-hop neighbourhood. The latency of every step of this procedure is measured in Section 5.11.

Complexity. A SEAL layer costs $\mathcal{O}(|\mathcal{V}|D^2 + |\mathcal{E}|D)$: the projections and mask MLPs are node-level matrix products, and each edge requires $\mathcal{O}(D)$ operations for its attention score, gate and message. Feature-mask generation adds $\mathcal{O}(|\mathcal{V}|dD)$. The explanation adds only a top- k selection at inference time. During training each epoch performs three forward passes (full, S and C), so an epoch costs about three times as much as for a SEAL network without the explanation objective.

Algorithm 1 RTXGNN training (as implemented in `rtxgnn/train.py` and `rtxgnn/objective.py`).

Require: graph $\mathcal{G}$ with features $\mathbf{X}$, timestamps; labelled training nodes $\mathcal{V}_{tr}$ with labels y ; validation nodes $\mathcal{V}_{val}$; hyper-parameters $\lambda_{suf}, \lambda_{nec}, \lambda_{sp}, k_f, k_e, e_w, E$, patience Q

Ensure: parameters θ^* and decision threshold η^*

- 1: $\mathcal{G}_{tr} \leftarrow$ subgraph induced by the 2-hop neighbourhoods of $\mathcal{V}_{tr}$; $\mathcal{G}_{ev} \leftarrow$ same for $\mathcal{V}_{val}$
 - 2: compute Δ_{uv} for all directed edges and $\mathbf{c}_v$ for all nodes (Eqs. 1-3); $\pi \leftarrow$ mean of y over $\mathcal{V}_{tr}$
 - 3: initialise θ ; Adam optimiser (learning rate $5 \cdot 10^{-3}$, weight decay 10^{-5})
 - 4: **for** $e = 0, \dots, E - 1$ **do**
 - 5: $\mathbf{M}^f, \{M^{n,(\ell)}, M^{e,(\ell)}\}_{\ell}, \hat{\mathbf{y}} \leftarrow$ forward pass of Eqs. 4-11 on $\mathcal{G}_{tr}$
 - 6: $\mathcal{L} \leftarrow \mathcal{L}_{pred}(\hat{\mathbf{y}}_{\mathcal{V}_{tr}}, y)$ {Eq. 13}
 - 7: $w \leftarrow \min(1, e/e_w)$
 - 8: **if** $w > 0$ **then**
 - 9: $S \leftarrow$ hard top- k_f features per node and top- k_e incoming edges per node and layer (straight-through); $C \leftarrow 1 - S$
 - 10: $\hat{\mathbf{y}}^S \leftarrow$ forward pass with masks replaced by S ; $\hat{\mathbf{y}}^C \leftarrow$ forward pass with masks replaced by C
 - 11: $\mathcal{L} \leftarrow \mathcal{L} + w \lambda_{suf} \mathcal{L}_{suf} + w \lambda_{nec} \mathcal{L}_{nec}$ {Eqs. 14, 15}
 - 12: **end if**
 - 13: $\mathcal{L} \leftarrow \mathcal{L} + w \lambda_{sp} \mathcal{L}_{sp}$ {Eq. 16}
 - 14: $\theta \leftarrow \theta - \text{Adam}(\nabla_{\theta} \mathcal{L})$ with gradient-norm clipping at 1
 - 15: **if** e is even **then**
 - 16: compute average precision AP_{val} of the forward pass on $\mathcal{G}_{ev}$ at the nodes $\mathcal{V}_{val}$
 - 17: **if** AP_{val} improved **then** $\theta^* \leftarrow \theta$ **else if** no improvement for Q epochs **then break**
 - 18: **end if**
 - 19: **end for**
 - 20: $\eta^* \leftarrow \arg \max_{\eta} \text{F1}_{val}(\eta)$ for the model θ^*
 - 21: **return** θ^*, η^*
-

Implementation. RTXGNN is implemented in PyTorch (Paszke et al., 2019) with PyTorch Geometric (Fey et al., 2025). We use $L = 2$ SEAL layers, hidden size $D = 64$, $H = 4$ heads, dropout 0.2, at most 300 epochs, early

Algorithm 2 RTXGNN inference for a request (as implemented in `rtxgnn/serving.py`).

Require: trained θ^* , threshold η^* ; CSR adjacency of the live graph; target transactions B ; fan-out $F = 25$; k_f, k_e

- 1: $N_0 \leftarrow B$; $\mathcal{E}_B \leftarrow \emptyset$
 - 2: **for** hop $h = 1, 2$ **do**
 - 3: for every $v \in N_{h-1}$ draw at most F incoming edges uniformly at random; add them to $\mathcal{E}_B$
 - 4: $N_h \leftarrow$ sources of the edges drawn in this hop
 - 5: **end for**
 - 6: build the subgraph on $N_0 \cup N_1 \cup N_2$ with edges $\mathcal{E}_B$ (features, Δ_{uv} , types, $\mathbf{c}_v$)
 - 7: one forward pass (Eqs. 4–11) without gradients
 - 8: **for** $v \in B$ **do**
 - 9: score $\hat{y}_v$; decision $\mathbb{K}[\hat{y}_v \geq \eta^*]$; explanation $\mathcal{X}_v$ from Eq. 12
 - 10: **end for**
 - 11: **return** scores, decisions and explanations
-

stopping with patience $Q = 40$ epochs on validation average precision (evaluated every second epoch), and gradient clipping at norm 1. Hardware is described in Section 4.4.

4. Experimental setup

All numbers in this and the following sections are produced by the scripts in the `experiments/` directory of the public repository. Each script writes one JSON record per run, and the tables are generated from these records.

4.1. Datasets

Elliptic Bitcoin dataset. The Elliptic dataset (Weber et al., 2019) contains 203,769 Bitcoin transactions (nodes) and 234,355 directed payment flows (edges) in 49 time steps of about two weeks each. 4,545 transactions (2.2%) are labelled illicit, 42,019 (20.6%) licit and the remaining 157,205 (77.1%) are unlabelled. Each transaction has 166 anonymised features. The first is the time step. The next 93 are *local* features of the transaction (the dataset documentation mentions, among others, the numbers of inputs and outputs, fees and output volume). The last 72 are *aggregated* features, statistics over

the transaction’s one-hop neighbourhood. The time step is not used as a feature, so every model receives $d = 165$ features. No edge crosses a time step. We use a temporal split: steps 1–30 for training (26,905 labelled transactions, 2,954 illicit), steps 31–34 for validation (2,989; 508 illicit) and steps 35–49 for testing (16,670; 1,083 illicit). [Appendix A](#) (Table [A.18](#)) lists the number of licit, illicit and unlabelled transactions in every time step.

T-Finance. T-Finance ([Tang et al., 2022](#)) is a transaction network of a financial company with 39,357 accounts, 21.2 million undirected edges (transactions between accounts) and 10 account features; 4.58% of the accounts are labelled fraudulent by human experts. We use the version and the first five fully supervised splits (40%/20%/40% train/validation/test) of GADBench ([Tang et al., 2023](#)). T-Finance has no timestamps, so it tests whether SEAL and the explanation objective carry over to a second real-world financial network with different features, entities and label source. HRAPE is switched off and temporal baselines are not applicable. To keep full-batch training tractable in memory, graph models see at most 10 uniformly sampled neighbours per account, which reduces the graph to about 0.4 million undirected edges.

Synthetic temporal laundering benchmark. The synthetic benchmark contains a signal that is known by construction and that only a model using timing information can exploit. The generator (`rtxgmn/synthetic.py`) works as follows. (1) A Barabási–Albert graph on $n = 5,000$ accounts ($m = 2$) defines which pairs transact. Each pair gets a random direction and a Poisson number of transactions (mean 3), with timestamps uniform on $[0, 180]$ days and log-normal amounts ($\mu = 5, \sigma = 1$). (2) Fifty laundering rings are laid on disjoint accounts. Each is a directed cycle of length $L \sim \mathcal{U}\{4, \dots, 8\}$ executed within a *burst* of 2 days at a random start time, with a 1–3% fee per hop. Ring members keep their ordinary transactions. (3) Fifty *decoy* cycles with the same length and amount distribution are laid on other accounts, but their hops are spread over the whole horizon. The cycle structure alone therefore does not identify laundering; the timing of consecutive hops does. (4) Every account has 16 features: 7 behavioural statistics computed from all its transactions (in/out counts, mean and standard deviation of in/out log-amounts, number of counterparties; standardised), a constant, and 8 i.i.d. $\mathcal{N}(0, 1)$ noise features. Ring members form the positive class (6.1% of accounts), and the ground-truth explanation of a ring member consists of its

two ring transactions. All accounts are scored at $t = 180$ days with a stratified random 60/20/20 split. Besides this *default* setting we generate four variants: *no_decoy* (no decoy cycles, so structure alone suffices), *wide_burst* (rings completed within 10 instead of 2 days), *label_noise* (5% of labels flipped) and *shift* (+0.5 added to the noise features of ring members, a weak feature signal). Every setting is generated with seeds 0–4, and seed s is also the model seed. By default the features carry no class signal, so the task cannot be solved from the features alone.

4.2. Preprocessing

For all neural models, each feature is mapped to a standard normal distribution by a quantile transform fitted on the transactions of the training steps only (Elliptic: steps 1–30, labelled and unlabelled; T-Finance: training accounts). No labels are used in this fit. Elliptic features are heavy-tailed, and the transform improves every neural model substantially (Table D.19 in Appendix D reports results with the raw features). The transform was adopted during development, when test results were also computed. To rule out a choice that fits only the test period, we checked it on the validation steps, which are used for model selection: the transform raises the validation AP of 5 of the 6 neural models and leaves that of the sixth (SEFraud) practically unchanged. Tree ensembles and logistic regression use the raw features. The transform does not help trees, which are invariant to monotone transformations, and on Elliptic its quantisation slightly reduced the random forest’s score.

4.3. Baselines

We compare against 16 baselines in four groups. Appendix B describes how each one is adapted.

- **Feature-only models:** logistic regression (LR, class-balanced), random forest (RF; 100 trees, 50 features per split, the configuration of Weber et al., 2019) and XGBoost (Chen and Guestrin, 2016) (300 trees, depth 6, class-balanced), and a two-layer MLP.
- **General GNNs:** GCN (Kipf and Welling, 2017), GraphSAGE (Hamilton et al., 2017) and GAT (Veličković et al., 2018).
- **Temporal GNNs:** EvolveGCN-O (Pareja et al., 2020), TGAT (Xu et al., 2020), TGN (Rossi et al., 2020) and APAN (Wang et al., 2021).

- **Fraud-specific GNNs:** CARE-GNN (Dou et al., 2020), PC-GNN (Liu et al., 2021), GAS (Li et al., 2019), FRAUDRE (Zhang et al., 2021), and the self-explaining SEFraud (Li et al., 2024).

All graph baselines were re-implemented in the same code base, because the original implementations target multi-relation review graphs (CARE-GNN, PC-GNN, FRAUDRE), heterogeneous graphs (GAS, SEFraud) or link prediction on continuous-time graphs (TGAT, TGN, APAN). The re-implementations follow the published architectures, adapted to the single-relation Elliptic graph. We cannot guarantee that they reach the performance of the authors’ own code in the original settings.

DySAT is not included because its temporal attention is vacuous on Elliptic (each node appears in a single snapshot; Section 2). H2-FDetector and DGA-GNN, which were compared in the first version of this study under a different protocol, are not included either. H2-FDetector separates homophilic from heterophilic connections within each relation of a multi-relation graph, and DGA-GNN groups non-additive raw attributes by decision-tree binning. On Elliptic’s single-relation graph with engineered features, their central mechanisms are represented by the label-aware neighbour selection of CARE-GNN and PC-GNN, the node-neighbour differences of FRAUDRE, and the tree ensembles, which are all included.

4.4. Protocol and statistical analysis

Training. All neural models use the same protocol and budget: hidden size 64, two message-passing layers where applicable, dropout 0.2, the class-balanced loss of Eq. (13) (plus the method’s own auxiliary losses), Adam with learning rate $5 \cdot 10^{-3}$ and weight decay 10^{-5} , gradient clipping at norm 1, at most 300 full-batch epochs, and early stopping on validation average precision with a patience of 40 epochs. No model, including RTXGNN, was tuned on the test data. In the main comparison no model received a per-model hyperparameter search. To check that the conclusions do not depend on this choice, we also ran an equal-budget tuning study (paragraph “Hyperparameter tuning” below).

Decision threshold. Every model outputs a score. The decision threshold is the one that maximises F1 on the validation set, and it is then applied unchanged to the test set. We also report threshold-free metrics: the area under the ROC curve (AUC) and average precision (AP, the area under the

precision–recall curve). AP is the more informative of the two under class imbalance.

Metrics. We report F1, precision and recall of the illicit class, together with AUC and AP. The overall test F1 is computed *once over all 16,670 labelled test transactions pooled together* (micro-averaging over time steps), as in [Weber et al. \(2019\)](#). It is not the mean of the per-step F1 scores. Section 5.6 explains why the two differ.

Seeds and tests. Each configuration on Elliptic is run with 10 random seeds (0–9), which change the parameter initialisation, dropout masks and, for the random forest and XGBoost, the bootstrap and column sampling. We report mean $\pm$ standard deviation over the seeds. Logistic regression is deterministic. We compare RTXGNN with every baseline with a two-sided Wilcoxon signed-rank test on the per-seed F1 and AP pairs ([Demšar, 2006](#)). Seeds are paired because both models use the same seed. We apply Holm’s correction ([Holm, 1979](#)) over the 16 comparisons of each metric. Ablations use 5 seeds (3 for the secondary variants), T-Finance uses GADBench splits 0–4, and the synthetic benchmark uses 5 generated graphs per setting. With 5 or 3 pairs, a two-sided Wilcoxon test cannot reach $p < 0.05$, so the ablations report paired t -tests. These p -values are not corrected for multiple comparisons and should be read as indicative.

Hyperparameter tuning. For RTXGNN and ten baselines, we ran a random search with the same budget for every model: 20 configurations drawn from a model-specific search space ([Appendix C](#)), the first being the default configuration of the main comparison. The ten baselines are the two tree ensembles, the three general GNNs, the continuous-time GNNs TGAT and TGN, and the fraud-specific GAS, FRAUDRE and SEFraud. They include the strongest model of every group and the other self-explaining model. Each configuration is trained with seeds 0–2, and the configuration with the highest mean validation AP is selected. Test results were recorded but never used for the selection. The selected configuration of every model is then trained with seeds 0–9 and compared with the selected configuration of RTXGNN as in the main comparison, with Holm’s correction over the 10 comparisons. The tuning study ran on the GPU machines (graph models) and on the CPU machine (tree ensembles).

Hardware. Experiments were distributed over two kinds of machines: a cloud virtual machine with 4 vCPUs of an Intel Xeon processor (Intel Xeon Processor @ 2.80GHz) and 15 GB of memory (CPU only), and Google Colab virtual machines with an NVIDIA T4 GPU. In the main comparison, seeds 0–2 and 5–9 ran on the CPU machine (except RTXGNN with seed 2) and seeds 3–4 on the GPU. The primary ablation study, the synthetic benchmark and the explanation study ran on the CPU machine. The EvolveGCN runs, the label-efficiency, retraining and T-Finance experiments, and the secondary ablations ran on the GPU. Each secondary ablation is compared with a reference model trained on the same hardware. Because the studies use different seed sets and hardware, the F1 of the full RTXGNN differs between them: 0.700 in the main comparison (10 seeds), 0.696 in the primary ablation (5 seeds, CPU), 0.724 in the secondary ablation (3 seeds, GPU) and 0.718 with all labels in the label-efficiency study (5 seeds, GPU). Each study is compared only with its own reference. Both machines used PyTorch 2 and PyTorch Geometric 2 without mixed precision, and the algorithm is identical on both. All latency measurements (Section 5.11) were taken on the CPU machine.

5. Results

5.1. Detection performance on *Elliptic*

Table 3 reports the test results of all models over 10 seeds.

RTXGNN reaches a test F1 of 0.700 ± 0.045 and an AP of 0.771. Among the twelve graph neural networks, only GAS obtains a practically identical F1 (0.701), and FRAUDRE and GAS obtain a practically identical AP (0.775 and 0.773). RTXGNN is ahead of all other GNNs on F1, including the general GNNs (GraphSAGE 0.684, GCN 0.690, GAT 0.646), the temporal GNNs (TGN 0.685, TGAT 0.673) and the fraud-specific models. In particular, RTXGNN improves on the other self-explaining model, SEFraud (+0.054 F1, $p = 0.010$ uncorrected; +0.029 AP, $p = 0.006$). After Holm correction over the 16 comparisons, the F1 differences to the GNN baselines are no longer significant except for PC-GNN, while the AP differences to GCN, GAT, CARE-GNN and PC-GNN remain significant (Table 3).

Two feature-only models remain strong competitors on *Elliptic*’s engineered features. XGBoost obtains the highest F1 (0.790) and AP (0.792), significantly above RTXGNN. The random forest reaches F1 0.733 and the same AP as RTXGNN (0.770), but its F1 varies considerably with the seed-dependent threshold (± 0.05). The strength of tree ensembles on these fea-

Table 3: Test results on Elliptic (time steps 35–49; 16,670 labelled transactions, 1,083 illicit), mean $\pm$ standard deviation over 10 seeds (logistic regression is deterministic). F1, precision and recall refer to the illicit class at the threshold chosen on the validation steps; AUC and AP are threshold-free. Best value per column in bold. $^\dagger/\ddagger$: RTXGNN significantly better/worse (two-sided Wilcoxon signed-rank test over the paired seeds, Holm-corrected over the 16 comparisons, $\alpha = 0.05$); marked for F1 and AP only.

Model	F1	Precision	Recall	AUC	AP
<i>Feature-only models</i>					
Logistic regression	$0.326 \pm 0.004^\dagger$	0.204 ± 0.004	0.809 ± 0.017	0.861 ± 0.000	$0.218 \pm 0.000^\dagger$
Random forest	0.733 ± 0.051	0.755 ± 0.115	0.721 ± 0.009	0.902 ± 0.004	0.770 ± 0.001
XGBoost	$0.790 \pm 0.015^\ddagger$	0.862 ± 0.040	0.729 ± 0.003	0.919 ± 0.006	$0.792 \pm 0.003^\ddagger$
MLP	$0.595 \pm 0.046^\dagger$	0.506 ± 0.068	0.733 ± 0.014	0.910 ± 0.010	0.760 ± 0.015
<i>General GNNs</i>					
GCN	0.690 ± 0.012	0.698 ± 0.048	0.685 ± 0.023	0.897 ± 0.011	$0.732 \pm 0.010^\dagger$
GraphSAGE	0.684 ± 0.034	0.654 ± 0.063	0.721 ± 0.011	0.908 ± 0.007	0.762 ± 0.014
GAT	0.646 ± 0.061	0.597 ± 0.106	0.717 ± 0.013	0.900 ± 0.010	$0.744 \pm 0.010^\dagger$
<i>Temporal GNNs</i>					
EvolveGCN-O	$0.332 \pm 0.170^\dagger$	0.285 ± 0.178	0.619 ± 0.244	0.765 ± 0.093	$0.374 \pm 0.192^\dagger$
TGAT	0.673 ± 0.040	0.632 ± 0.077	0.727 ± 0.013	0.919 ± 0.007	0.770 ± 0.011
TGN	0.685 ± 0.036	0.665 ± 0.070	0.712 ± 0.008	0.917 ± 0.010	0.759 ± 0.009
APAN	0.646 ± 0.057	0.585 ± 0.084	0.729 ± 0.010	0.907 ± 0.006	0.762 ± 0.016
<i>Fraud-specific GNNs</i>					
CARE-GNN	0.670 ± 0.031	0.621 ± 0.058	0.732 ± 0.011	0.912 ± 0.006	$0.756 \pm 0.015^\dagger$
PC-GNN	$0.600 \pm 0.029^\dagger$	0.530 ± 0.044	0.696 ± 0.024	0.906 ± 0.014	$0.665 \pm 0.031^\dagger$
GAS	0.701 ± 0.035	0.683 ± 0.072	0.725 ± 0.007	0.915 ± 0.008	0.773 ± 0.009
FRAUDRE	0.668 ± 0.040	0.621 ± 0.074	0.730 ± 0.010	0.920 ± 0.008	0.775 ± 0.006
SEFraud	0.646 ± 0.025	0.584 ± 0.042	0.724 ± 0.005	0.910 ± 0.013	0.742 ± 0.022
<i>Proposed</i>					
RTXGNN (ours)	0.700 ± 0.045	0.680 ± 0.088	0.728 ± 0.008	0.905 ± 0.009	0.771 ± 0.008

tures agrees with [Weber et al. \(2019\)](#) and with the general finding of [Grinsztajn et al. \(2022\)](#). The Elliptic features already contain 72 aggregates of the one-hop neighbourhood, so much of the graph information is available to a feature-only model. The trees do not produce explanations over the relations between transactions, which is the purpose of RTXGNN. RTXGNN is clearly ahead of the MLP, which sees the same features without message passing (+0.105 F1).

EvolveGCN-O is the weakest graph model (0.332). Its recurrent weight evolution has to be carried from the training period through all test snapshots without supervision. Because no transaction appears in more than one snapshot, the model cannot exploit the node-level continuity for which it was designed. The continuous-time models TGN and TGAT perform similarly to static GNNs, for the reasons discussed in Section 2.

Table 4: Equal-budget tuning study on Elliptic. Every model uses the configuration with the highest mean validation AP among 20 configurations (seeds 0–2) and is then evaluated with 10 seeds (mean $\pm$ standard deviation). F1 (default): the main comparison (Table 3); Δ F1: change due to tuning. $^\dagger/^\ddagger$: tuned RTXGNN significantly better/worse (two-sided Wilcoxon signed-rank test, Holm-corrected over the 10 comparisons, $\alpha = 0.05$).

Model	F1 (default)	F1	Precision	Recall	AP	Δ F1
XGBoost	0.790	$0.789 \pm 0.012^\ddagger$	0.857 ± 0.032	0.732 ± 0.003	$0.794 \pm 0.001^\ddagger$	-0.000
Random forest	0.733	$0.769 \pm 0.032^\ddagger$	0.826 ± 0.081	0.724 ± 0.008	$0.788 \pm 0.001^\ddagger$	+0.036
GCN	0.690	0.710 ± 0.016	0.735 ± 0.041	0.689 ± 0.012	$0.748 \pm 0.007^\dagger$	+0.021
GraphSAGE	0.684	0.704 ± 0.035	0.690 ± 0.067	0.724 ± 0.008	0.766 ± 0.011	+0.021
GAT	0.646	0.711 ± 0.018	0.708 ± 0.042	0.716 ± 0.008	0.769 ± 0.004	+0.065
TGAT	0.673	0.698 ± 0.028	0.670 ± 0.051	0.730 ± 0.007	0.783 ± 0.006	+0.024
TGN	0.685	0.705 ± 0.025	0.699 ± 0.056	0.714 ± 0.012	0.773 ± 0.009	+0.020
GAS	0.701	0.704 ± 0.024	0.687 ± 0.048	0.723 ± 0.004	0.776 ± 0.005	+0.003
FRAUDRE	0.668	0.684 ± 0.024	0.647 ± 0.045	0.726 ± 0.004	0.775 ± 0.003	+0.015
SEFraud	0.646	0.671 ± 0.043	0.624 ± 0.084	0.733 ± 0.015	0.774 ± 0.011	+0.025
RTXGNN (ours)	0.700	0.703 ± 0.045	0.686 ± 0.084	0.726 ± 0.008	0.775 ± 0.010	+0.003

5.2. Robustness to hyperparameter tuning

The main comparison uses one configuration for all neural models. Table 4 shows what happens when every model receives the same tuning budget, with selection on the validation steps only. Tuning improves most baselines, most strongly GAT (+0.065 F1), the random forest (+0.036) and SEFraud (+0.025). The configuration selected for RTXGNN reaches F1 0.703 ± 0.045 , essentially its default score (+0.003). The overall picture does not change. After Holm correction, XGBoost (F1 0.789) and the random forest (0.769) remain significantly more accurate than RTXGNN. The tuned GNNs reach F1 between 0.671 and 0.711, and none of their F1 differences to RTXGNN is significant. RTXGNN is on average ahead of the tuned SEFraud (+0.032) and FRAUDRE (+0.020), and within 0.01 F1 of the other GNNs. On AP, RTXGNN is significantly better than the tuned GCN and not significantly different from the other GNNs. The conclusions of Section 5.1 therefore do not depend on the choice of hyperparameters.

5.3. What the scores mean operationally

An F1 around 0.7 still means that errors are frequent. At the validation-chosen threshold, RTXGNN raises on average 1,188 alerts in the test period, misses 293 of the 1,083 illicit transactions (27%), and 399 of its alerts (32%) concern licit transactions (Table 5). As a stand-alone decision rule this would not be acceptable. The ranking, however, is highly informative for review queues with limited capacity: among the 500 highest-scored test transactions,

Table 5: Operational view of the test period (1,083 illicit among 16,670 labelled transactions), mean over the seeds. Left: number of alerts, missed illicit transactions and false alerts at the validation-chosen threshold. Right: precision among the top- K scored test transactions for alert budgets K .

Model	at validation threshold			precision at an alert budget of			
	alerts	missed	false alerts	250	500	1,000	2,000
RTXGNN (ours)	1,188	293	399	1.00	0.99	0.77	0.41
GraphSAGE	1,206	303	425	1.00	0.99	0.76	0.41
GAT	1,344	306	568	0.99	0.97	0.75	0.41
SEFraud	1,350	297	565	0.98	0.96	0.74	0.41
Random forest	1,058	302	277	1.00	1.00	0.78	0.40
XGBoost	919	293	129	1.00	1.00	0.80	0.41

0.99 of those flagged by RTXGNN are illicit, and among the top 1,000 the proportion is 0.77. The differences between the strongest models appear mainly at the level of the automatic threshold, where XGBoost produces fewer false alerts. For a decision maker, the relevant choice is therefore the size of the review queue. The threshold should be set by the capacity of the investigation team and by the relative cost of a missed case and a false alert, not by maximising F1.

5.4. Ablation study

Table 6 isolates the components of RTXGNN. The primary study uses the same CPU runs and seeds 0–4 as Table 3. The secondary study adds further variants with 3 seeds, each compared with a reference model trained on the same hardware.

Temporal encoding. Replacing HRAPE by Time2Vec on the same time gaps lowers F1 by -0.067 (paired t -test, $p = 0.04$, uncorrected), and Time2Vec on absolute time lowers it by -0.040 ($p = 0.02$). Adding an absolute-time encoding to HRAPE lowers the mean F1 by -0.049 and makes it much less stable across seeds (standard deviation 0.08 against 0.05). Removing the temporal encoding altogether costs -0.030 F1, which is not significant with 5 seeds ($p = 0.26$). This is consistent with Section 3.2: on Elliptic HRAPE mainly contributes the velocity (degree) signal, because all edges connect transactions of the same time step. The design choice that matters most is to encode time relative to the scored transaction rather than absolutely. This choice was made during development, when test results were also computed.

Table 6: Ablation of RTXGNN on Elliptic (mean $\pm$ standard deviation; Δ F1 is the mean paired difference to the full model over the common seeds, with the p -value of a paired t -test). Variants listed with 3 seeds are secondary.

Variant	seeds	F1	AP	Δ F1 (paired p)
<i>Primary study (5 seeds)</i>				
RTXGNN (full)	5	0.696 ± 0.048	0.769 ± 0.009	–
without HRAPE	5	0.666 ± 0.035	0.769 ± 0.018	-0.030 (0.26)
HRAPE $\rightarrow$ Time2Vec (gaps)	5	0.629 ± 0.028	0.766 ± 0.010	-0.067 (0.04)
HRAPE + absolute-time encoding	5	0.647 ± 0.080	0.773 ± 0.009	-0.049 (0.33)
without SEAL masks (plain attention)	5	0.704 ± 0.028	0.777 ± 0.013	+0.008 (0.75)
without $\mathcal{L}_{sp}$	5	0.718 ± 0.018	0.776 ± 0.015	+0.021 (0.30)
without $\mathcal{L}_{suf}$ and $\mathcal{L}_{nec}$	5	0.708 ± 0.050	0.773 ± 0.007	+0.012 (0.72)
<i>Secondary study (3 seeds, trained on GPU; own reference)</i>				
RTXGNN (full)	3	0.724 ± 0.031	0.771 ± 0.012	–
Time2Vec on absolute time	3	0.685 ± 0.033	0.770 ± 0.014	-0.040 (0.02)
without $\mathcal{L}_{suf}$	3	0.658 ± 0.015	0.765 ± 0.006	-0.067 (0.06)
without $\mathcal{L}_{nec}$	3	0.728 ± 0.049	0.780 ± 0.003	+0.003 (0.89)
without curriculum	3	0.666 ± 0.063	0.755 ± 0.020	-0.059 (0.39)
focal loss instead of balanced BCE	3	0.599 ± 0.051	0.742 ± 0.010	-0.125 (0.08)

The validation steps alone lead to the same choice. HRAPE reaches a validation AP of 0.961 (validation F1 0.921), against 0.946 (0.882) with Time2Vec on the same time gaps and 0.951 (0.894) with the additional absolute-time encoding, and its validation F1 is higher in 5 of 5 and 3 of 5 paired seeds, respectively. The absolute-time encoding also makes the validation AP less stable across seeds (standard deviation 0.026 against 0.011). In the secondary study, Time2Vec on absolute time reaches a validation AP of 0.953, against 0.970 for HRAPE.

Explanation components. Replacing the SEAL masks by plain attention (+0.008, $p = 0.75$), removing the sparsity term (+0.021, $p = 0.30$), or removing both fidelity terms (+0.012, $p = 0.72$) does not change detection performance significantly. In the secondary study, removing the sufficiency term alone lowers F1 by -0.067 ($p = 0.06$), and removing the necessity term leaves it unchanged. The secondary result is not confirmed by the primary study, in which removing both terms leaves F1 unchanged (+0.012). The full model is also not more accurate than its variants without masks or without explanation terms. All these differences are within the variation across seeds. The explanation machinery thus comes at no measurable cost in accuracy, but it does not improve accuracy either. What it provides is an explanation with

Table 7: Mask sparsity compared with dropout and weight decay (Elliptic; notation as in Table 6).

Variant	seeds	F1	AP	Δ F1 (paired p)
<i>Primary study (5 seeds)</i>				
masks + $\mathcal{L}_{sp}$ ($\lambda_{sp} = 0.05$), dropout 0.2, wd 10^{-5}	5	0.696 ± 0.048	0.769 ± 0.009	–
masks, no $\mathcal{L}_{sp}$	5	0.718 ± 0.018	0.776 ± 0.015	+0.021 (0.30)
<i>Secondary study (3 seeds, trained on GPU; own reference)</i>				
masks + $\mathcal{L}_{sp}$ ($\lambda_{sp} = 0.05$), dropout 0.2, wd 10^{-5}	3	0.724 ± 0.031	0.771 ± 0.012	–
masks + $\mathcal{L}_{sp}$ ($\lambda_{sp} = 0.2$)	3	0.645 ± 0.003	0.767 ± 0.017	-0.080 (0.04)
masks, no $\mathcal{L}_{sp}$, dropout 0	3	0.722 ± 0.038	0.762 ± 0.019	-0.003 (0.91)
masks, no $\mathcal{L}_{sp}$, dropout 0.5	3	0.725 ± 0.024	0.778 ± 0.013	+0.001 (0.98)
masks, no $\mathcal{L}_{sp}$, weight decay 10^{-3}	3	0.724 ± 0.017	0.782 ± 0.006	-0.001 (0.97)

the properties measured in Section 5.8, which the variants without masks or without fidelity terms do not have.

Training choices. Class-balanced cross-entropy is clearly preferable to focal loss (-0.125 F1 with focal loss), and the curriculum on the explanation terms tends to help (-0.059 without it, not significant). The model is also sensitive to its capacity (Table 8). Hidden size 64 with two SEAL layers, the configuration fixed before the experiments, performs best among the tested settings, and a single layer is significantly worse (-0.111 F1). The tuning study (Section 5.2), which also tries three layers, selects three layers on the validation steps, but this does not change the test F1.

5.5. Mask sparsity versus conventional regularisation

Table 7 compares the mask sparsity term with conventional regularisers. With the default weight $\lambda_{sp} = 0.05$, the model performs like models trained without the sparsity term but with dropout 0, dropout 0.5 or weight decay 10^{-3} . All differences are at most 0.003 F1 in the secondary study and 0.021 in the primary study, and none is significant. A four times larger weight ($\lambda_{sp} = 0.2$) lowers F1 by -0.080 ($p = 0.04$). Mask sparsity is therefore not a better regulariser than the standard ones, but it is not a harmful one either at a moderate weight. Its purpose is to keep explanations selective (Section 5.12).

5.6. Temporal behaviour, drift and retraining

Table 9 and Figure 2 break the test period down by time step. The period has two clearly different phases. In steps 35–42, RTXGNN reaches a

Table 8: Sensitivity to the hidden size D and the number of SEAL layers L (secondary study, 3 seeds, GPU; notation as in Table 6).

Variant	seeds	F1	AP	Δ F1 (paired p)
<i>Secondary study (3 seeds, trained on GPU; own reference)</i>				
$D = 32$	3	0.606 ± 0.048	0.766 ± 0.010	-0.119 (0.09)
$D = 64$ (default)	3	0.724 ± 0.031	0.771 ± 0.012	–
$D = 128$	3	0.672 ± 0.035	0.760 ± 0.018	-0.053 (0.11)
$L = 1$ layer	3	0.613 ± 0.015	0.752 ± 0.014	-0.111 (0.03)

Table 9: Per-step test results on Elliptic. Static: models trained on steps 1–30 (mean $\pm$ s.d. over 10 seeds for RTXGNN; mean for the random forest). Retrained: refitted before each step on steps $\leq t - 5$, validated on $t - 4..t - 1$ (mean over 3 seeds).

Step	Licit	Illicit	RTXGNN, static			RF	RTXGNN / RF
			Precision	Recall	F1	F1 static	F1 retrained
35	1,159	182	0.92 ± 0.03	0.95 ± 0.01	0.93 ± 0.01	0.94	0.91 / 0.96
36	1,675	33	0.50 ± 0.13	0.95 ± 0.04	0.65 ± 0.10	0.85	0.58 / 0.91
37	458	40	0.82 ± 0.04	0.62 ± 0.03	0.70 ± 0.02	0.72	0.72 / 0.76
38	645	111	0.78 ± 0.04	0.91 ± 0.01	0.84 ± 0.02	0.90	0.93 / 0.94
39	1,102	81	0.73 ± 0.10	0.94 ± 0.01	0.82 ± 0.07	0.84	0.94 / 0.96
40	1,099	112	0.75 ± 0.09	0.69 ± 0.02	0.71 ± 0.04	0.71	0.77 / 0.78
41	1,016	116	0.82 ± 0.08	0.93 ± 0.01	0.87 ± 0.04	0.89	0.94 / 0.95
42	1,915	239	0.82 ± 0.08	0.81 ± 0.01	0.81 ± 0.04	0.83	0.87 / 0.88
43	1,346	24	0.00 ± 0.01	0.00 ± 0.01	0.00 ± 0.01	0.00	0.00 / 0.00
44	1,567	24	0.02 ± 0.01	0.05 ± 0.02	0.03 ± 0.01	0.03	0.04 / 0.07
45	1,216	5	0.02 ± 0.03	0.10 ± 0.14	0.03 ± 0.05	0.00	0.00 / 0.00
46	710	2	0.06 ± 0.05	0.45 ± 0.16	0.11 ± 0.08	0.11	0.00 / 0.33
47	824	22	0.00	0.00	0.00	0.00	0.08 / 0.23
48	435	36	0.00	0.00	0.00	0.00	0.09 / 0.14
49	420	56	0.05 ± 0.04	0.01 ± 0.01	0.02 ± 0.02	0.03	0.45 / 0.50

per-step F1 between 0.66 and 0.94 (mean 0.79), with recall above 0.8 in most steps, and the random forest behaves very similarly. From step 43 onwards all models fail: the mean per-step F1 of RTXGNN over steps 43–49 is 0.03, and that of the random forest is 0.02. [Weber et al. \(2019\)](#) attribute this collapse to the shutdown of a large dark market after step 42. At the same time, the number of illicit transactions per step falls from between 33 and 239 to between 2 and 56, so the late steps carry only 169 of the 1,083 illicit test transactions (16%).

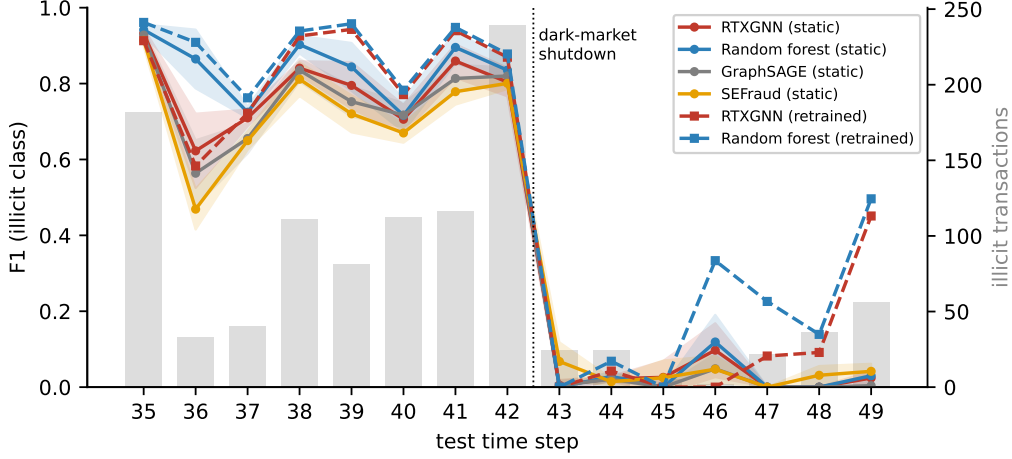


Figure 2: F1 per test time step (lines; shaded band: ± 1 s.d. over seeds) and number of illicit transactions per step (grey bars, right axis). Dashed lines: models retrained before every step with a one-step label delay.

This explains why a near-zero F1 in the late steps is compatible with a pooled test F1 of about 0.7. The pooled F1 is computed once over all test transactions and is therefore dominated by the 914 illicit transactions of steps 35–42. The unweighted mean of the fifteen per-step F1 scores of RTXGNN is only 0.436.

Retraining with a one-step label delay. To assess how much of the collapse a deployment could recover, we refit the models before every test step t on all labelled steps up to $t - 5$, validate and choose the threshold on steps $t - 4$ to $t - 1$, and score step t . Labels thus become available one step after the transactions occur. Neural models are warm-started from the previous step’s model. Retraining improves the early steps slightly (RTXGNN 0.83, random forest 0.89). In the late steps it raises the mean F1 of RTXGNN from 0.03 to 0.10 and that of the random forest from 0.02 to 0.18. The recovery is real but partial. Once the illicit activity changes, a model can only learn the new pattern after labelled examples of it exist, and the late steps contain very few of them. Retraining is therefore necessary but not sufficient, and a deployment also needs drift monitoring and complementary controls (Section 6.4).

Table 10: Test F1 on Elliptic with a stratified fraction of the training labels (mean $\pm$ s.d. over 5 subsets; in parentheses: percentage of the model’s own full-label F1). Validation and test sets are never subsampled.

Labels	train (illicit)	RTXGNN	Random forest	GAT	MLP
5%	1,346 (150)	0.593 ± 0.050 (83%)	0.614 ± 0.091 (82%)	0.564 ± 0.029 (86%)	0.582 ± 0.049 (98%)
10%	2,689 (297)	0.623 ± 0.022 (87%)	0.585 ± 0.069 (78%)	0.628 ± 0.068 (96%)	0.617 ± 0.033 (104%)
20%	5,382 (592)	0.650 ± 0.059 (91%)	0.694 ± 0.076 (93%)	0.660 ± 0.057 (101%)	0.584 ± 0.029 (98%)
50%	13,450 (1,478)	0.666 ± 0.048 (93%)	0.673 ± 0.063 (90%)	0.624 ± 0.061 (95%)	0.630 ± 0.020 (106%)
100%	26,905 (2,954)	0.718 ± 0.021 (100%)	0.748 ± 0.046 (100%)	0.657 ± 0.039 (100%)	0.595 ± 0.043 (100%)

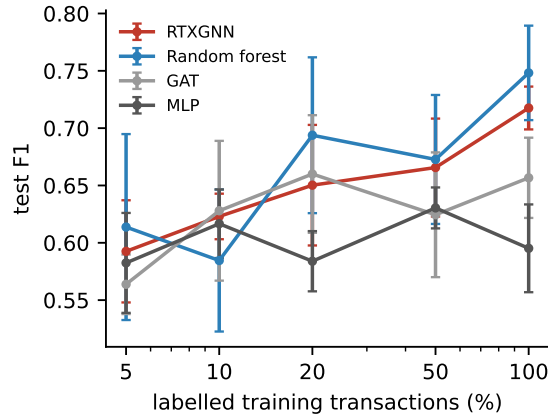


Figure 3: Label efficiency on Elliptic (mean $\pm$ s.d. over 5 stratified subsets).

5.7. Label efficiency

Table 10 and Figure 3 report the effect of training with a stratified fraction of the labels. Each fraction is sampled within every training time step and class and repeated with 5 subsets; the validation and test sets are never subsampled. RTXGNN’s F1 increases monotonically with the number of labels. With 10% of the labels (2,689 transactions, 297 illicit) it keeps 87% of its full-label F1 (0.623 versus 0.718), and with 50% it keeps 93%. The random forest is more sensitive to the particular subset: its standard deviation across subsets is larger than that of RTXGNN at every fraction below 100% (up to three times larger), and its F1 is not monotone in the label fraction. The MLP is insensitive to the number of labels but stays at a lower level throughout. Because the full-label runs of this experiment were trained on a GPU with 5 seeds, their F1 (0.718) differs slightly from Table 3.

Table 11: Feature explanations of 600 Elliptic test transactions (300 illicit, 300 licit), mean over 5 trained models. Fid^+ : drop of the predicted-class probability when the top- k features are removed (higher is better); Fid^- : drop when only the top- k features are kept (lower is better). Stab.: Jaccard similarity of the top-10 features under a perturbation of the input by $\mathcal{N}(0, 0.05^2)$. Time: milliseconds per explained transaction on 2 CPU threads, batched. Post-hoc explainers are applied to the same RTXGNN models.

Explainer	$\text{Fid}^+ \uparrow$			$\text{Fid}^- \downarrow$			Stab.	Time
	$k=5$	10	20	5	10	20	J@10	ms
SEAL masks (self-explaining)	0.015	0.032	0.062	0.143	0.076	0.047	0.96	0.41
Saliency (post hoc)	0.025	0.056	0.138	0.123	0.090	0.058	0.96	1.21
Integrated Gradients (post hoc)	0.082	0.142	0.227	0.111	0.069	0.039	0.96	41.12
GNNExplainer (post hoc)	0.159	0.279	0.353	-0.011	-0.032	-0.042	0.44	128.95
Random ranking	0.006	0.011	0.019	0.305	0.284	0.251	–	0.00
SEFraud masks (on SEFraud)	0.009	0.016	0.031	0.220	0.200	0.150	0.99	0.19
SEAL masks, trained w/o $\mathcal{L}_{suf}, \mathcal{L}_{nec}$	0.030	0.050	0.159	0.289	0.233	0.118	0.99	0.39
SEAL masks, trained w/o $\mathcal{L}_{sp}$	0.009	0.018	0.030	0.111	0.076	0.068	0.93	0.40

Table 12: Class-wise faithfulness at $k = 10$ (illicit and licit test transactions).

Explainer	illicit targets		licit targets	
	$\text{Fid}^+@10$	$\text{Fid}^-@10$	$\text{Fid}^+@10$	$\text{Fid}^-@10$
SEAL masks (self-explaining)	0.027	0.088	0.036	0.065
Saliency (post hoc)	0.067	0.097	0.045	0.084
Integrated Gradients (post hoc)	0.226	0.037	0.057	0.101
GNNExplainer (post hoc)	0.307	-0.014	0.251	-0.050
Random ranking	0.010	0.404	0.011	0.165
SEFraud masks (on SEFraud)	0.016	0.359	0.017	0.042
SEAL masks, trained w/o $\mathcal{L}_{suf}, \mathcal{L}_{nec}$	0.034	0.331	0.067	0.134
SEAL masks, trained w/o $\mathcal{L}_{sp}$	0.020	0.114	0.015	0.037

5.8. Faithfulness and cost of the explanations

Table 11 and Figure 4 compare the feature explanations of 600 test transactions (300 illicit, 300 licit) for five independently trained models. The post-hoc explainers are applied to the same RTXGNN models, so differences reflect the explainers, not the predictors.

Sufficiency. Keeping only the ten features selected by SEAL changes the predicted-class probability by 0.076 on average ($\text{Fid}^-@10$). This is better than saliency (0.090; at $k = 5$ saliency is slightly more sufficient) and close to Integrated Gradients (0.069), and it is far better than a random selection

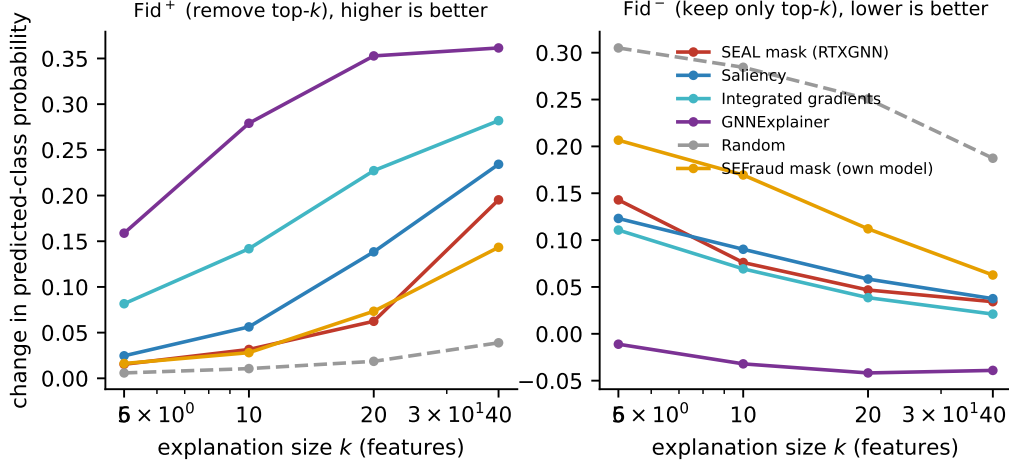


Figure 4: Sparsity-fidelity trade-off of feature explanations on Elliptic (mean over 5 models and 600 test transactions).

(0.284) and than SEFraud’s masks on SEFraud (0.200). The sufficiency term is responsible for this property. The same architecture trained without $\mathcal{L}_{suf}$ and $\mathcal{L}_{nec}$ yields masks with $\text{Fid}^-@10$ of 0.233, three times worse.

Necessity. Removing the ten features selected by SEAL lowers the predicted-class probability by only 0.032 ($\text{Fid}^+@10$). This is a clear weakness. Integrated Gradients (0.142) and in particular GNNExplainer (0.279) identify features whose removal changes the prediction much more. The Elliptic features are highly redundant (72 of them aggregate the local features of neighbours), and a model can often recover the same information from features outside a ten-feature explanation. The necessity term constrains the complement of the whole hard view, including the edge view, and it is not sufficient to make the top ten features individually indispensable. GNNExplainer’s high scores come at a price: it takes 128.95 ms per transaction, about 300 times more than reading SEAL’s masks (0.41 ms), and its explanations are unstable. Its top-10 features have a Jaccard similarity of only 0.44 under a small input perturbation, against 0.96 for SEAL. Integrated Gradients offers a middle ground at 41.12 ms.

Class-wise behaviour and agreement across models. Table 12 shows that SEAL’s sufficiency is similar for illicit and licit transactions, whereas Integrated Gradients and SEFraud’s masks are much more sufficient for one class than

Table 13: Test F1 on the synthetic benchmark (mean $\pm$ s.d. over 5 generated graphs per setting).

Model	default	no decoys	wide burst	5% label noise	feature shift
MLP	0.46 ± 0.04	0.63 ± 0.06	0.46 ± 0.04	0.41 ± 0.02	0.62 ± 0.02
Random forest	0.42 ± 0.03	0.65 ± 0.07	0.42 ± 0.03	0.37 ± 0.04	0.52 ± 0.05
XGBoost	0.45 ± 0.03	0.68 ± 0.07	0.45 ± 0.03	0.35 ± 0.06	0.60 ± 0.02
GCN	0.50 ± 0.08	0.73 ± 0.02	0.50 ± 0.08	0.39 ± 0.04	0.69 ± 0.04
GraphSAGE	0.52 ± 0.04	0.73 ± 0.06	0.52 ± 0.04	0.39 ± 0.05	0.68 ± 0.04
GAT	0.53 ± 0.08	0.68 ± 0.06	0.53 ± 0.08	0.37 ± 0.06	0.68 ± 0.04
TGAT	0.59 ± 0.07	0.67 ± 0.10	0.54 ± 0.05	0.35 ± 0.02	0.74 ± 0.04
TGN	0.44 ± 0.06	0.57 ± 0.05	0.43 ± 0.08	0.28 ± 0.06	0.56 ± 0.04
SEFraud	0.45 ± 0.05	0.73 ± 0.04	0.45 ± 0.05	0.36 ± 0.05	0.72 ± 0.03
RTXGNN w/o SEAL	0.55 ± 0.11	0.83 ± 0.07	0.56 ± 0.12	0.39 ± 0.03	0.80 ± 0.07
RTXGNN w/o HRAPE	0.49 ± 0.09	0.78 ± 0.10	0.49 ± 0.09	0.39 ± 0.05	0.77 ± 0.06
RTXGNN, Time2Vec	0.49 ± 0.05	0.74 ± 0.10	0.48 ± 0.05	0.40 ± 0.03	0.69 ± 0.07
RTXGNN	0.52 ± 0.09	0.83 ± 0.06	0.49 ± 0.10	0.38 ± 0.06	0.75 ± 0.10
edges / positive rate	30.7k / 6.0%	30.4k / 6.0%	30.7k / 6.0%	30.7k / 10.6%	30.7k / 6.0%

Table 14: Ground-truth evaluation of edge explanations on the synthetic benchmark: AUC of the edge scores for identifying the two ring transactions among the incoming edges of each correctly detected ring member (mean over graphs 0–2; up to 100 accounts per graph).

Setting	Model	SEAL importance	attention only	GNNExplainer	random
default	RTXGNN	0.72 ± 0.29	0.63 ± 0.19	0.70 ± 0.21	0.54 ± 0.03
default	RTXGNN w/o HRAPE	0.60 ± 0.18	0.84 ± 0.18	0.54 ± 0.11	0.50 ± 0.04
default	RTXGNN, Time2Vec	0.65 ± 0.27	0.59 ± 0.31	0.58 ± 0.20	0.49 ± 0.03
wide burst	RTXGNN	0.81 ± 0.23	0.69 ± 0.29	0.75 ± 0.24	0.55 ± 0.02
wide burst	RTXGNN w/o HRAPE	0.60 ± 0.18	0.84 ± 0.18	0.54 ± 0.11	0.50 ± 0.04
wide burst	RTXGNN, Time2Vec	0.70 ± 0.22	0.55 ± 0.28	0.62 ± 0.21	0.49 ± 0.02

for the other. Masks of independently trained RTXGNN models agree only weakly on the top-10 features (mean Jaccard similarity 0.12, against 0.03 for random rankings), which again reflects the redundancy of the features. An analyst should therefore read an explanation as the features the *deployed* model relied on, not as a model-independent property of the transaction.

In summary, SEAL provides explanations that are sufficient, stable and cheap at inference time, but weakly necessary. For the transactions an analyst actually investigates, a more expensive post-hoc explainer can be run in addition when a stronger counterfactual statement is needed.

Detected laundering ring (synthetic benchmark, seed 0)

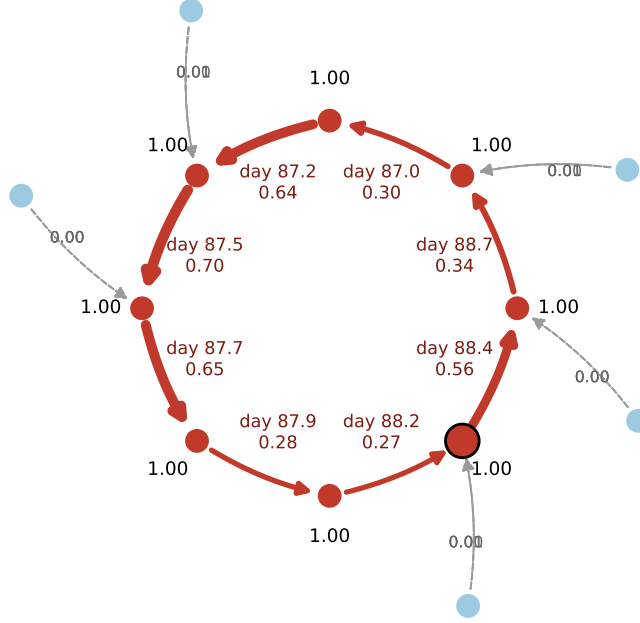


Figure 5: A laundering ring detected by RTXGNN on the synthetic benchmark (graph 0, default setting). Red: ring members with their predicted probability; blue: up to three ordinary counterparties per member; outlined: a test account. Red solid arrows are the ring transactions (labelled with the transaction day and the relative importance), grey dashed arrows ordinary transactions into ring members. Arrow width is the relative SEAL importance of a transaction, i.e. its share of the total importance $\sum_u I_{uv}$ of all transactions received by the member v (Eq. 12).

5.9. Controlled evaluation on the synthetic benchmark

The synthetic benchmark isolates the two signals that Elliptic cannot provide: laundering rings whose members have no distinguishing features, and decoy cycles that can only be told apart by the timing of their hops (Table 13). As intended, the task is hard. Feature-only models reach F1 below 0.5 in the default setting, since the features carry only indirect information (ring members have two extra transactions). Graph models improve on this.

The temporal mechanism helps where timing matters. In the default setting, RTXGNN (F1 0.52) is slightly ahead of its variants without HRAPE (0.49) and with Time2Vec on the same time gaps (0.49), but the differences are smaller than the standard deviations. TGAT, whose time encoding is applied to every message, is the strongest model in this setting (0.59). When decoys are absent, so that the cycle structure alone identifies the rings, RTXGNN and its variant without SEAL masks are the best models (0.83). The 5% label-noise setting is hard for all models (F1 about 0.4). The wide-burst setting leaves every static model unchanged, because only transaction times differ, which confirms that the static baselines cannot see timing.

Ground-truth evaluation of edge explanations. Because the planted ring transactions are known, the synthetic data allow a direct test of the edge explanations (Table 14). For each correctly detected ring member, we rank its incoming transactions by SEAL’s edge importance I_{uv} (Eq. 12) and compute the AUC for identifying its two ring transactions. SEAL’s importances reach an AUC of 0.72 in the default setting and 0.81 in the wide-burst setting. This is higher than GNNExplainer applied to the same models (0.70 and 0.75), than the raw attention weights (0.63 and 0.69), and than random scores (about 0.5). This ordering holds for the full model. For the variant without HRAPE, the raw attention weights identify the ring transactions better (0.84) than its edge importances (0.60). Over all 264 ring members detected by the seed-0 model, the most important incoming transaction is a ring transaction in 91% of the cases. Figure 5 shows one ring. All its hops happen within two days, they receive most of the importance of their receiving accounts, and the ordinary transactions of the same accounts receive almost none. The variation of the edge AUC across graphs is large (standard deviations of about 0.2–0.3), so these results show a clear tendency rather than a uniform effect.

5.10. Second real-world dataset: T-Finance

T-Finance is a transaction network between the accounts of a financial company, with account-level features and expert labels. Its graph is far denser than Elliptic’s, and it has no engineered neighbourhood aggregates. Table 15 shows that the picture differs from Elliptic accordingly. All graph neural networks clearly outperform the feature-only models: the best tree ensemble reaches F1 0.773 and AP 0.812, while the GNNs reach F1 between 0.801 and 0.849. RTXGNN, which runs here without its temporal component, is among

Table 15: Test results on T-Finance (GADBench splits 0–4, 40/20/40; mean $\pm$ s.d.). No timestamps are available, so HRAPE is disabled and temporal baselines are not applicable.

Model	F1	AUC	AP
Logistic regression	0.641 ± 0.028	0.924 ± 0.006	0.629 ± 0.041
Random forest	0.765 ± 0.010	0.939 ± 0.005	0.798 ± 0.009
XGBoost	0.773 ± 0.012	0.949 ± 0.002	0.812 ± 0.009
MLP	0.757 ± 0.026	0.955 ± 0.003	0.809 ± 0.013
GCN	0.801 ± 0.009	0.963 ± 0.003	0.850 ± 0.013
GraphSAGE	0.849 ± 0.010	0.969 ± 0.004	0.892 ± 0.008
GAT	0.829 ± 0.014	0.969 ± 0.002	0.882 ± 0.011
CARE-GNN	0.825 ± 0.016	0.961 ± 0.007	0.868 ± 0.016
PC-GNN	0.823 ± 0.010	0.965 ± 0.005	0.870 ± 0.012
FRAUDRE	0.842 ± 0.009	0.969 ± 0.002	0.886 ± 0.009
SEFraud	0.834 ± 0.010	0.973 ± 0.001	0.890 ± 0.008
RTXGNN (ours)	0.837 ± 0.011	0.972 ± 0.003	0.889 ± 0.009

the best models (F1 0.837, AUC 0.972, AP 0.889). It is within about one point of F1 of the best model (GraphSAGE, F1 0.849, AP 0.892) and on par with FRAUDRE (0.842) and the self-explaining SEFraud (0.834). SEAL’s explanation mechanism therefore transfers to a second real financial network, with different entities, features and label source, without loss of accuracy.

5.11. End-to-end latency

Table 16 reports the end-to-end latency of scoring requests on one CPU core. A request samples the 2-hop neighbourhood of its transactions, builds the subgraph, runs the model and returns the explanation. For a single transaction, RTXGNN needs 2.8 ms at the median and 5.9 ms at the 99th percentile. Of this, 0.7 ms is neighbourhood sampling, 2.1 ms the forward pass and 0.24 ms the extraction of the explanation from the masks. The explanation therefore adds less than a millisecond. Producing the explanation with GNNExplainer instead raises the median latency to 582.0 ms and the 99th percentile to 1023.9 ms, two orders of magnitude more. For batches of 256 transactions, a single core scores and explains 2,013 transactions per second (125.6 ms per batch at the median). RTXGNN is slower than the plain GNNs (GAT: 1.5 ms for one transaction) because of its mask generators and its wider messages, but it stays far below the 10–100 ms budgets of online screening. Using four threads changes the single-transaction latency very little (2.8 ms at the median), because the computation is too small to

Table 16: End-to-end latency per request of B transactions on CPU (Intel Xeon Processor @ 2.80GHz), in milliseconds: percentiles of the total and mean time of each stage (one thread; sample: 2-hop neighbourhood sampling with fan-out 25 and subgraph construction; forward: model; explain: reading top-5 features and top-3 edges from the masks, or running GNNExplainer for 100 iterations). tx/s: throughput. MLP and GCN are timed with untrained weights, which does not affect their latency. Peak resident memory of the process, including the in-memory graph: 1,048 MB.

Model	B	p50	p95	p99	sample	forward	explain	tx/s
MLP	1	0.7	1.0	2.8	0.6	0.2	0.0	1,315
MLP	32	3.1	4.2	5.1	2.4	0.8	0.0	9,890
MLP	256	16.1	18.7	21.2	12.0	4.3	0.0	15,635
GCN	1	1.2	1.5	1.7	0.5	0.7	0.0	789
GCN	32	5.1	6.7	9.7	2.4	2.8	0.0	6,161
GCN	256	27.0	35.6	42.3	13.5	14.6	0.0	9,113
GAT	1	1.5	2.6	2.9	0.6	1.0	0.0	594
GAT	32	7.3	10.8	12.4	2.9	4.7	0.0	4,207
GAT	256	35.1	46.2	55.5	14.1	22.1	0.0	7,064
GAT + GNNExplainer	1	305.3	352.7	369.7	0.9	1.1	307.0	3
SEFraud	1	1.7	2.8	4.0	0.6	1.2	0.0	542
SEFraud	32	11.0	15.6	19.3	2.7	8.3	0.1	2,852
SEFraud	256	60.6	71.7	84.4	14.5	46.7	0.8	4,125
RTXGNN (ours)	1	2.8	4.7	5.9	0.7	2.1	0.2	336
RTXGNN (ours)	32	21.7	31.7	38.3	2.9	16.3	3.5	1,413
RTXGNN (ours)	256	125.6	149.2	158.6	13.5	86.6	27.1	2,013
RTXGNN (ours) + GNNExplainer	1	582.0	959.1	1023.9	0.9	2.2	635.7	2

parallelise, and improves the throughput for large batches (2,752 transactions per second). The peak resident memory of the serving process, including the complete in-memory test graph, was 1,048 MB.

These figures do not include the retrieval of the neighbourhood from a production data store, which we emulate with an in-memory CSR graph, or network overhead. They are therefore lower bounds for a deployed system. They are also not directly comparable with the 0.4ms per transaction reported for SEFraud (Li et al., 2024), which refers to GPU inference amortised over large batches. Under our protocol, SEFraud (1.7 ms for one transaction) is somewhat faster than RTXGNN, since it has no node masks and no temporal encoding.

5.12. Case studies

Table 17 shows the explanations of four test transactions: two correctly detected illicit transactions and one correctly classified licit transaction from

Table 17: Explanations of four Elliptic test transactions by RTXGNN (seed 0). Open features: number of features with mask value above 0.5 (in parentheses: local ones). The first five open features are listed in the order of the mask generator’s pre-sigmoid score; groups: L = local, A = aggregated (the only semantics documented for the anonymised features). Score in parentheses: after removing these five features. Neighbours: the most important incoming and outgoing transactions (edge importance I_{uv}) with their labels.

Case	score (w/o top-5)	open fea- tures (lo- cal)	first five open features: index (group)	top neighbours: direction, label, importance
true positive, step 42	1.00 (0.99)	82 (44)	162 (A), 28 (L), 160 (A), 89 (L), 84 (L)	out., unlabelled, 0.00; inc., licit, 0.00
true positive, step 42	1.00 (1.00)	79 (41)	28 (L), 162 (A), 160 (A), 89 (L), 84 (L)	out., unlabelled, 0.23; inc., illicit, 0.02
true negative, step 42	0.01 (0.02)	75 (41)	162 (A), 160 (A), 88 (L), 28 (L), 120 (A)	out., unlabelled, 0.00; out., unlabelled, 0.00; inc., licit, 0.00
false negative (after step 43), step 44	0.01 (0.01)	77 (44)	120 (A), 141 (A), 160 (A), 88 (L), 28 (L)	out., unlabelled, 0.00

step 42, and one illicit transaction from step 44 that RTXGNN misses. Because Elliptic’s features are anonymised, we report feature indices together with their documented group (local or aggregated) and do not attach any further meaning to them.

Three observations follow from these cases and from the statistics of the masks over all test transactions. First, the feature mask is nearly binary. For a typical test transaction, about 30% of the features are fully open (gate > 0.99), about 30% fully closed (< 0.01), and 75–82 features are open in the four cases, roughly half of them local. The mask therefore acts as a per-transaction feature selection rather than as a fine-grained ranking. The ordering within the open set in Table 17 is by the mask generator’s pre-sigmoid score. Second, the leading features are largely shared across transactions, i.e. the selection is mostly global. Third, removing the first five open features changes the scores by at most 0.005, in line with the low necessity found in Section 5.8. The missed transaction after step 43 receives a confident licit score, and its explanation offers no warning sign. This illustrates that explanations do not help with new patterns that the model has never seen. For decision support, the most useful elements are thus (i) the retained feature set, which documents which inputs could have influenced the score, (ii) the neighbouring transactions with non-negligible importance, which point the

analyst to related cases, and (iii) the score itself, used to rank the review queue.

6. Discussion

6.1. What the detection results mean for decision making

Under a common protocol, RTXGNN is among the most accurate graph models on Elliptic, and gradient-boosted trees are more accurate still. The differences between the best GNNs are of the same order as the variation across seeds. An F1 of about 0.7 means, at the validation-chosen threshold, that about a quarter of the illicit test transactions are missed and that about a third of the alerts are false (Section 5.3). No financial institution would run such a model as an autonomous decision maker. The realistic use is as a prioritisation tool in a human review queue, where the operating point is set by the capacity of the investigation team (Table 5) and the model is combined with rules and other controls. In this role, three properties matter beyond F1: ranking quality at the top of the list (AP), the effort needed to review an alert, and robustness over time. The contribution of RTXGNN is mainly the second one. It provides a sufficient and stable attribution with every score at a small additional inference cost (0.24 ms per single request), while its detection performance is on par with the strongest GNNs.

6.2. Explanations, regulation and how masks could be operationalised

The masks do not by themselves make a system compliant with GDPR Article 22 or the ECOA. The regulations cited in Section 2 impose obligations on institutions and processes: lawful basis and human involvement for automated decisions, notices with specific reasons, documented and validated models, and auditable AML decisions. No model architecture can satisfy these obligations by itself. Several of them also do not concern the Elliptic setting at all. The ECOA, for instance, governs consumer credit decisions, not the screening of cryptocurrency transactions. A ranking of anonymous feature indices, as produced on Elliptic, is not a meaningful explanation for a data subject, an analyst or a regulator.

The masks can nevertheless support such processes when the institution controls the feature definitions. We see four concrete requirements.

1. **A governed feature dictionary.** Every model input needs a documented, business-level definition, for example “number of distinct counterparties in the last 7 days”. Features derived from the graph also need

a description of how they are computed. Top- k feature attributions can then be translated into candidate reason statements. This is impossible on Elliptic, whose features are anonymised, and is the main reason we only report feature *groups* (local versus aggregated) in the case studies.

2. **Reason codes curated by experts.** Attributions indicate which inputs the score depended on and in which direction the prediction moves when they are removed. They do not justify the decision. A curated mapping from (groups of) features to reason codes, reviewed by compliance officers, and a rule for which reasons are reported (for example the top reasons that push the score towards “illicit”) are needed before any attribution reaches a customer notice or a suspicious-activity report.
3. **Human review and audit trail.** The score, the threshold version, the model version, and the feature and edge attributions of every alert can be logged without a separate explanation step, because they are produced by the forward pass that produced the score. Such a log supports the documentation expected in AML investigations and in model-risk management ([Board of Governors of the Federal Reserve System and Office of the Comptroller of the Currency, 2011](#)). The final decision remains with the analyst.
4. **Validation of the explanations themselves.** Before explanations are used in any customer-facing or supervisory context, their faithfulness, their stability and their usefulness to analysts must be validated. We report faithfulness and stability for RTXGNN (Section 5.8). Its explanations are stable under small input perturbations but agree only weakly across independently retrained models, so explanations must be versioned together with the model that produced them. Usefulness to analysts requires a user study with practising analysts, which we have not conducted.

Under these conditions, the masks are best regarded as evidence that can support transparency obligations. They do not demonstrate compliance with GDPR, ECOA, FATF guidance, MiCA or the EU AI Act ([European Parliament and Council of the European Union, 2024](#)).

6.3. *Mask sparsity is regularisation, not a new kind of regularisation*

The sparsity term is an L_1 penalty on auxiliary gate activations, and a claim about its regularising effect must be tested against standard regularisers. Our controlled comparison (Section 5.5) does not show a large effect.

With 5 seeds, the differences between mask sparsity, stronger dropout and stronger weight decay are within the seed-to-seed variation. We therefore make only the weaker claim that the explanation objective does not cost detection performance. We do not claim that it improves it.

6.4. *Deployment: drift, retraining and latency*

All models collapse on Elliptic after time step 43. This is not specific to graph models or to RTXGNN. [Weber et al. \(2019\)](#) attribute it to the shutdown of a large dark market, after which the illicit activity in the data changes. The retraining experiment (Section 5.6) shows that refitting with a one-step label delay recovers part of the lost performance for both RTXGNN and the random forest. It cannot anticipate a new illicit pattern before labels for it exist. A deployment would therefore need drift monitoring on the score distribution and on the alert yield, periodic retraining with a documented schedule, and fallback rules for known typologies. Our latency measurements (Section 5.11) show that the model itself fits comfortably in an online budget on a single CPU core. In practice the budget is dominated by the retrieval of the transaction neighbourhood from the institution’s data store, which we emulate with an in-memory graph. Latency in a production system will also depend on that store.

6.5. *Limitations*

- **Data.** Elliptic’s snapshots contain no cross-snapshot edges and no intra-snapshot timestamps, so the temporal components of RTXGNN are close to inactive on it. T-Finance has no timestamps at all. The evidence for HRAPE therefore rests on its comparison with Time2Vec on Elliptic and on synthetic data. Validation on a real transaction dataset with continuous timestamps and repeated account activity is the most important next step.
- **Baselines.** All graph baselines are our re-implementations under one protocol and budget, and some were adapted from multi-relation or continuous-time settings. They may perform better with their original code. The equal-budget tuning study (Section 5.2) covers ten of the 16 baselines with 20 configurations each; a larger search could change individual results.
- **Explanations.** Faithfulness is measured with perturbation metrics that replace features by the training mean. Such perturbations can

move inputs off the data manifold. Our training objective also uses perturbations of this kind, which may favour RTXGNN on these metrics. We mitigate this by applying the post-hoc explainers to the same trained model and by reporting a random-explanation reference. The feature masks are nearly binary and weakly necessary, so they identify a set of features the prediction may depend on rather than a ranked list of indispensable ones. No user study has been carried out.

- **Compute.** All experiments were run on a 4-vCPU machine and a single T4 GPU. Training cost is about three times that of the same network without the explanation objective. Scalability to graphs with billions of edges was not evaluated.

7. Conclusions

We presented RTXGNN, a self-explaining temporal graph neural network for fraud detection. Its masks gate message passing and are returned as the explanation of every score, and its training objective makes the top- k explanation sufficient and its complement uninformative. The temporal encoding (HRAPE) uses only relative time. In a controlled comparison with 16 baselines over 10 seeds on Elliptic, RTXGNN matches the best graph neural network (F1 0.700, AP 0.771) and is ahead of the self-explaining SEFraud on average. Its ranking is precise at the top of the alert queue, which is where analysts spend their effort. Gradient-boosted trees on Elliptic’s engineered features remain more accurate (F1 0.790). An equal-budget hyperparameter search for RTXGNN and ten baselines leaves this ranking unchanged. They do not, however, produce explanations over the relations between transactions. RTXGNN’s explanations come with every score, without a separate explainer and at a small cost. They are nearly as sufficient as Integrated Gradients and much more stable than GNNExplainer, and on synthetic data they single out the planted laundering transactions. A single transaction is scored and explained end to end in 5.9 ms (99th percentile) on one CPU core.

The evaluation also shows the limits of the approach. All models fail after the regime change at time step 43, and only retraining on newly labelled data recovers part of the performance. The explanations are weakly necessary: removing the top features rarely changes the prediction, because Elliptic’s features are redundant. HRAPE carries little information on Elliptic’s snapshots and is unavailable on T-Finance, so its benefit rests mainly

on comparisons with Time2Vec and on synthetic evidence. Finally, the explanations support, but do not by themselves establish, the transparency expected by regulation. That requires governed feature definitions, curated reason codes, human review and validation with analysts.

Future work should (i) evaluate the temporal components on real transaction data with continuous timestamps and repeated account activity, (ii) study continual-learning and drift-detection strategies that shorten the recovery after regime changes, (iii) conduct a user study with fraud analysts on the usefulness of the explanations, and (iv) assess the robustness of models and explanations against adversaries who know the explanation mechanism.

Data and code availability

The implementation, the experiment scripts, the per-run result records and the scripts that generate every table and figure of this paper are available at https://github.com/vinhqdang/explainable_GNN_financial_fraud. The Elliptic dataset is available from its publishers (Weber et al., 2019). T-Finance is distributed with GADBench (Tang et al., 2023). The synthetic benchmark is generated by `rtxgnn/synthetic.py`.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author(s) used ChatGPT in order to polish the writing. After using this tool/service, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the published article.

Appendix A. Elliptic: class counts per time step

Appendix B. Adaptation of the baselines

All baselines use the shared protocol of Section 4.4 (hidden size 64, two layers, dropout 0.2, class-balanced loss, Adam, early stopping on validation AP, threshold tuned on validation). The code is in `rtxgnn/baselines.py`.

- **GCN, GraphSAGE, GAT.** PyTorch Geometric layers on the symmetrised payment graph; GAT uses 4 heads.

Table A.18: Labelled and unlabelled transactions per time step of Elliptic and the split each step belongs to.

Step	Split	Licit	Illicit	Unlab.	Step	Split	Licit	Illicit	Unlab.
1	train	2,130	17	5,733	26	train	421	96	2,006
2	train	1,099	18	3,427	27	train	182	24	883
3	train	1,268	11	5,342	28	train	199	85	1,369
4	train	1,410	30	4,253	29	train	845	329	3,101
5	train	1,874	8	4,921	30	train	441	83	1,959
6	train	480	5	3,843	31	val	604	106	2,106
7	train	1,101	102	4,845	32	val	981	342	3,202
8	train	1,098	67	3,292	33	val	418	23	2,710
9	train	530	248	4,218	34	val	478	37	1,971
10	train	954	18	5,755	35	test	1,159	182	4,166
11	train	565	131	3,600	36	test	1,675	33	4,685
12	train	490	16	1,541	37	test	458	40	2,808
13	train	518	291	3,719	38	test	645	111	2,135
14	train	374	43	1,605	39	test	1,102	81	1,577
15	train	471	147	3,021	40	test	1,099	112	3,270
16	train	402	128	2,445	41	test	1,016	116	4,210
17	train	712	99	2,574	42	test	1,915	239	4,986
18	train	337	52	1,587	43	test	1,346	24	3,693
19	train	665	80	2,761	44	test	1,567	24	3,384
20	train	640	260	3,391	45	test	1,216	5	4,377
21	train	541	100	2,896	46	test	710	2	2,807
22	train	1,605	158	4,131	47	test	824	22	4,275
23	train	1,134	53	2,978	48	test	435	36	2,483
24	train	989	137	3,466	49	test	420	56	1,978
25	train	476	118	1,720					
Total		42,019	4,545	157,205					

- **EvolveGCN-O.** The weight matrix of each of the two GCN layers is the hidden state of an LSTM whose input is the previous weight matrix (rows are treated as the batch). The LSTM steps once per time step over all 49 snapshots, including validation and test snapshots, which are processed without labels. Each snapshot is convolved with its own normalised adjacency.
- **TGAT.** Two temporal attention layers. Keys and values are built from $[\mathbf{h}_u \parallel \Phi(\Delta_{uv}) \parallel r_{uv}]$ and queries from $[\mathbf{h}_v \parallel \Phi(0)]$, with the functional time encoding $\Phi(\Delta) = \cos(\omega\Delta + \mathbf{b})$ (Xu et al., 2020). All neighbours are used, since Elliptic degrees are small.

- **TGN.** A GRU memory is updated from messages $[\mathbf{s}_v \parallel \mathbf{s}_u \parallel \Phi(t - t_v^-) \parallel \mathbf{x}_u]$ aggregated by their mean. The embedding module is one TGAT layer over $\text{ReLU}(\mathbf{W}\mathbf{x} + \mathbf{W}'\mathbf{s})$. Events are processed in chronological batches: one batch per time step on Elliptic, and one batch per day on the synthetic data. Embeddings of the nodes involved in a batch are computed from the memory after the batch update. On Elliptic every node takes part in the events of a single time step only, so the memory before that step is always empty.
- **APAN.** Every node has a mailbox of its $K = 10$ most recent mails. Its embedding is multi-head attention of its own state over the mailbox followed by an MLP. Mails $\mathbf{W}[\mathbf{z}_u \parallel \mathbf{z}_v \parallel \Phi(\Delta_{uv})]$ are propagated to the 1-hop neighbours. On snapshot data we run one encoding pass, one propagation round and a second encoding pass per step, instead of the asynchronous propagation of the original system (Wang et al., 2021).
- **CARE-GNN.** Single-relation version: a label-aware similarity network (a linear classifier per layer, trained with an auxiliary cross-entropy of weight 2) defines the distance $|p_u - p_v|$. Each node keeps the top- p fraction of its neighbours with the smallest distance, and p is adapted after every epoch by ± 0.02 when the average distance of the selected neighbours decreases or increases (the reinforcement-learning rule of Dou et al., 2020). Aggregation is $\text{ReLU}(\mathbf{W}[\mathbf{h}_v \parallel \text{mean}_{u \in \text{sel}} \mathbf{h}_u])$.
- **PC-GNN.** Pick: in every epoch the loss is computed on a label-balanced sample of training nodes, drawn with probability proportional to $(\deg(v) + 1)/|\text{class}(v)|$, with four times as many nodes as there are illicit training nodes. Choose: neighbours whose label-aware distance exceeds $\rho = 0.5$ are dropped, and every illicit training node receives two additional neighbours drawn from the illicit training nodes. Label-aware predictor with auxiliary loss weight 2.
- **GAS.** Two GCN branches, one on the payment graph and one on a k -nearest-neighbour graph ($k = 5$, cosine similarity of features) built within each time step, concatenated before the classifier. This replaces the review-content similarity graph of the original.
- **FRAUDRE.** Layers $\text{ReLU}(\mathbf{W}[\mathbf{h}_v \parallel \text{mean}_u \mathbf{h}_u \parallel |\text{mean}_u \mathbf{h}_u - \mathbf{h}_v|])$ (fraud-aware convolution with node-neighbour differences). The classifier

takes the concatenation of the input and of all layer outputs, and imbalance is handled by the class-balanced loss.

- **SEFraud.** A feature-mask MLP and an edge-mask MLP gate a two-layer graph transformer, as in Li et al. (2024) but on a single node and edge type. Its loss adds to the classification loss a triplet loss (margin 1, weight 0.5) with anchor the embedding of the unmasked graph, positive the embedding of the masked graph and negative the embedding of the complementary graph, plus an L_1 mask penalty of weight 0.01.

Appendix C. Search spaces of the tuning study

Every model receives 20 configurations: its default configuration (Section 4.4) and 19 configurations drawn uniformly at random from the grid below (`experiments/run_tuning.py`). Each configuration is trained with seeds 0–2 and selected by its mean validation AP.

- **RTXGNN:** hidden size $\{64, 128\}$, SEAL layers $\{2, 3\}$, dropout $\{0, 0.2, 0.3, 0.5\}$, learning rate $\{1, 2, 5, 10\} \cdot 10^{-3}$, weight decay $\{0, 10^{-5}, 10^{-4}, 10^{-3}\}$, $\lambda_{suf}, \lambda_{nec} \in \{0.25, 0.5, 1\}$, $\lambda_{sp} \in \{0.01, 0.05\}$, $k_f \in \{10, 20\}$.
- **GCN, GraphSAGE, GAT:** hidden size $\{32, 64, 128\}$, layers $\{2, 3\}$, dropout, learning rate and weight decay as for RTXGNN.
- **TGAT, TGN, GAS, FRAUDRE, SEFraud:** hidden size $\{32, 64, 128\}$, dropout, learning rate and weight decay as for RTXGNN.
- **XGBoost:** trees $\{300, 600, 1000\}$, depth $\{4, 6, 8\}$, learning rate $\{0.03, 0.05, 0.1\}$, row subsampling $\{0.6, 0.8, 1\}$, column subsampling $\{0.5, 0.8, 1\}$, minimum child weight $\{1, 5, 10\}$.
- **Random forest:** trees $\{100, 300, 500\}$, features per split $\{20, 50, 100, \sqrt{d}\}$, minimum leaf size $\{1, 2, 5\}$, class weights $\{\text{none}, \text{balanced per bootstrap sample}\}$.

The selected configuration of RTXGNN has three SEAL layers, hidden size 64, dropout 0.2, learning rate $2 \cdot 10^{-3}$, no weight decay, $\lambda_{suf} = 0.5$, $\lambda_{nec} = 1$, $\lambda_{sp} = 0.01$ and $k_f = 10$.

Appendix D. Effect of feature preprocessing

Table D.19: Test results on Elliptic (seed 0) with raw features versus quantile-normalised features (Section 4). All other settings are identical (the raw-feature runs were trained on the GPU machine). Validation AP is computed on steps 31–34, which are used for model selection.

Model	raw features			quantile-normalised		
	val. AP	test F1	test AP	val. AP	test F1	test AP
MLP	0.839	0.553	0.500	0.930	0.625	0.745
GCN	0.872	0.527	0.441	0.964	0.692	0.726
GraphSAGE	0.939	0.537	0.508	0.977	0.711	0.766
GAT	0.900	0.544	0.539	0.963	0.721	0.748
SEFraud	0.959	0.514	0.487	0.957	0.651	0.729
RTXGNN (ours)	0.950	0.530	0.522	0.964	0.664	0.759

References

- Afriyie, J.K., Tawiah, K., Pels, W.A., Addai-Henne, S., Dwamena, H.A., Owiredun, E.O., Ayeh, S.A., Eshun, J., 2023. A supervised machine learning algorithm for detecting and predicting fraud in credit card transactions. *Decision Analytics Journal* 6, 100163.
- Agarwal, C., Queen, O., Lakkaraju, H., Zitnik, M., 2023. Evaluating explainability for graph neural networks. *Scientific Data (Nature)* 10. doi:[10.1038/s41597-023-01974-x](https://doi.org/10.1038/s41597-023-01974-x).
- Amara, K., Ying, R., Zhang, Z., Han, Z., Shan, Y., Brandes, U., Schemm, S., Zhang, C., 2022. GraphFramEx: Towards systematic evaluation of explainability methods for graph neural networks, in: *Learning on Graphs Conference (LoG)*.
- Asiri, A., Somasundaram, K., 2025. Graph convolution network for fraud detection in bitcoin transactions. *Scientific Reports* 15, 11076.
- Baldassarre, F., Azizpour, H., 2019. Explainability techniques for graph convolutional networks, in: *ICML Workshop on Learning and Reasoning with Graph-Structured Data*. ArXiv:1905.13686.
- Board of Governors of the Federal Reserve System, Office of the Comptroller of the Currency, 2011. Supervisory guidance on model risk management (SR letter 11-7). <https://www.federalreserve.gov/supervisionreg/srletters/sr1107.htm>.

- Breiman, L., 2001. Random forests. *Machine Learning* 45, 5–32.
- Cao, S., Yang, X., Chen, C., Zhou, J., Li, X., Qi, Y., 2019. TitAnt: Online real-time transaction fraud detection in ant financial. *Proceedings of the VLDB Endowment* 12, 2082–2093. doi:[10.14778/3352063.3352126](https://doi.org/10.14778/3352063.3352126).
- Chen, T., Guestrin, C., 2016. XGBoost: A scalable tree boosting system, in: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794.
- Cherif, S., et al., 2024. Financial fraud detection using graph neural networks: A systematic review. *Expert Systems with Applications* 238, 121660. doi:[10.1016/j.eswa.2023.121660](https://doi.org/10.1016/j.eswa.2023.121660).
- Demšar, J., 2006. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research* 7, 1–30.
- Dong, C., Xiao, S., 2025. Enhancing financial fraud detection in digital finance applications through machine learning algorithms and real-time data analytics. *Journal of Computational Methods in Sciences and Engineering* , 14727978251352131.
- Dou, Y., Liu, Z., Sun, L., Deng, Y., Peng, H., Yu, P.S., 2020. Enhancing graph neural network-based fraud detectors against camouflaged fraudsters, in: *Proceedings of the 29th ACM International Conference on Information and Knowledge Management (CIKM)*, pp. 315–324. doi:[10.1145/3340531.3411903](https://doi.org/10.1145/3340531.3411903). arXiv:2008.08692.
- Duan, M., Zheng, T., Gao, Y., Wang, G., Feng, Z., Wang, X., 2024. DGA-GNN: Dynamic grouping aggregation GNN for fraud detection, in: *Proceedings of the 38th AAAI Conference on Artificial Intelligence (AAAI)*, pp. 11914–11922. doi:[10.1609/aaai.v38i10.29067](https://doi.org/10.1609/aaai.v38i10.29067).
- European Parliament and Council of the European Union, 2024. Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (artificial intelligence act). *Official Journal of the European Union*, L series, 12 July 2024.
- Faber, L., Moghaddam, A.K., Wattenhofer, R., 2021. When comparing to ground truth is wrong: On evaluating GNN explanation methods, in: *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, pp. 332–341. doi:[10.1145/3447548.3467283](https://doi.org/10.1145/3447548.3467283).

- Fey, M., Sunil, J., Nitta, A., Puri, R., Shah, M., Stojanovic, B., Bendias, R., Barghi, A., Kocijan, V., Zhang, Z., He, X., Lenssen, J.E., Leskovec, J., 2025. PyG 2.0: Scalable learning on real world graphs. CoRR abs/2507.16991.
- Fu, Z., Shi, X., Liang, T., Chen, Z., Sun, Y., 2026. Phishing fraud identity inference based on temporal transaction graph attention network. *The Computer Journal* , 145.
- Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., Bouchachia, A., 2014. A survey on concept drift adaptation. *ACM Computing Surveys* 46, 44:1–44:37.
- Goodman, B., Flaxman, S., 2016. European union regulations on algorithmic decision-making and a ‘right to explanation’, in: *ICML Workshop on Human Interpretability in Machine Learning (WHI)*. ArXiv:1606.08813.
- Grinsztajn, L., Oyallon, E., Varoquaux, G., 2022. Why do tree-based models still outperform deep learning on typical tabular data?, in: *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.
- Hamilton, W.L., Ying, Z., Leskovec, J., 2017. Inductive representation learning on large graphs, in: *NIPS*, pp. 1024–1034.
- Holm, S., 1979. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics* 6, 65–70.
- Huang, Q., Yamada, M., Tian, Y., Singh, D., Chang, Y., 2023. Graphlime: Local interpretable model explanations for graph neural networks. *IEEE Trans. Knowl. Data Eng.* 35, 6968–6972.
- Jacob, K., Nagarajan, D., et al., 2025. A golden eagle-based hybrid deep learning model for automobile insurance fraud detection. *Decision Analytics Journal* , 100619.
- Kazemi, S.M., Goel, R., Eghbali, S., Ramanan, J., Sahota, J., Thakur, S., Wu, S., Smyth, C., Poupart, P., Brubaker, M., 2019. Time2Vec: Learning a vector representation of time. arXiv preprint arXiv:1907.05321 .

- Kipf, T.N., Welling, M., 2017. Semi-supervised classification with graph convolutional networks, in: ICLR (Poster), OpenReview.net.
- Li, A., Qin, Z., Liu, R., Yang, Y., Li, D., 2019. Spam review detection with graph convolutional networks, in: Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM), pp. 2703–2711.
- Li, K., Liu, Y., Ao, X., Chi, J., Feng, J., Yang, H., He, Q., 2024. SEFraud: Graph-based self-explainable fraud detection via interpretative mask learning, in: Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD), pp. 1723–1734. doi:[10.1145/3637528.3671534](https://doi.org/10.1145/3637528.3671534). arXiv:2406.11389.
- Lin, T.Y., Goyal, P., Girshick, R., He, K., Dollár, P., 2017. Focal loss for dense object detection, in: Proceedings of the IEEE international conference on computer vision, pp. 2980–2988.
- Lipton, Z.C., 2018. The mythos of model interpretability. Communications of the ACM 61, 36–43.
- Liu, Y., Ao, X., Qin, Z., Chi, J., Feng, J., Yang, H., He, Q., 2021. Pick and choose: A GNN-based imbalanced learning approach for fraud detection, in: Proceedings of the Web Conference 2021 (WWW), pp. 3168–3177. doi:[10.1145/3442381.3449989](https://doi.org/10.1145/3442381.3449989).
- Liu, Z., Chen, C., Li, L., Zhou, J., Li, X., Song, L., Qi, Y., 2019. GeniePath: Graph neural networks with adaptive receptive paths, in: Proceedings of the 33rd AAAI Conference on Artificial Intelligence (AAAI), pp. 4424–4431. doi:[10.1609/aaai.v33i01.33014424](https://doi.org/10.1609/aaai.v33i01.33014424). arXiv:1802.00910.
- Liu, Z., Chen, C., Yang, X., Zhou, J., Li, X., Song, L., 2018. Heterogeneous graph neural networks for malicious account detection, in: Proceedings of the 27th ACM International Conference on Information and Knowledge Management (CIKM), pp. 2077–2085. doi:[10.1145/3269206.3272010](https://doi.org/10.1145/3269206.3272010). arXiv:2002.12307.
- Liu, Z., Dou, Y., Yu, P.S., Deng, Y., Peng, H., 2020. Alleviating the inconsistency problem of applying graph neural network to fraud detection,

- in: Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR), pp. 1569–1572. doi:[10.1145/3397271.3401253](https://doi.org/10.1145/3397271.3401253). arXiv:2005.00625.
- Lu, M., Han, Z., Rao, S.X., Zhang, Z., Zhao, Y., Shan, Y., Raghunathan, R., Zhang, C., Jiang, J., 2022. BRIGHT - graph neural networks in real-time fraud detection, in: Proceedings of the 31st ACM International Conference on Information and Knowledge Management (CIKM), pp. 4598–4607. doi:[10.1145/3511808.3557136](https://doi.org/10.1145/3511808.3557136). arXiv:2205.13084.
- Lucic, A., ter Hoeve, M., Tolomei, G., de Rijke, M., Silvestri, F., 2022. CF-GNNExplainer: Counterfactual explanations for graph neural networks, in: Proceedings of the 25th International Conference on Artificial Intelligence and Statistics (AISTATS), PMLR. pp. 6829–6854. ArXiv:2102.03322.
- Luo, D., Cheng, W., Xu, D., Yu, W., Zong, B., Chen, H., Zhang, X., 2020. Parameterized explainer for graph neural network, in: Advances in Neural Information Processing Systems 33 (NeurIPS), pp. 19620–19631. ArXiv:2011.04573.
- Miao, S., Liu, M., Li, P., 2022. Interpretable and generalizable graph learning via stochastic attention mechanism, in: International Conference on Machine Learning (ICML), pp. 15524–15543.
- Pareja, A., Domeniconi, G., Chen, J., Ma, T., Suzumura, T., Kanezashi, H., Kaler, T., Schardl, T.B., Leiserson, C.E., 2020. EvolveGCN: Evolving graph convolutional networks for dynamic graphs, in: Proceedings of the 34th AAAI Conference on Artificial Intelligence (AAAI), pp. 5363–5370. doi:[10.1609/aaai.v34i04.5984](https://doi.org/10.1609/aaai.v34i04.5984). arXiv:1902.10191.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., et al., 2019. Pytorch: An imperative style, high-performance deep learning library. Advances in neural information processing systems 32.
- Pope, P.E., Kolouri, S., Rostami, M., Martin, C.E., Hoffmann, H., 2019. Explainability methods for graph convolutional neural networks, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 10772–10781. doi:[10.1109/CVPR.2019.01103](https://doi.org/10.1109/CVPR.2019.01103).

- Rao, S.X., Jiang, J., Han, Z., Yin, H., 2025. Fraud detection in e-commerce: A systematic review of transaction risk prevention. *Anomaly Detection- Methods, Complexities and Applications* .
- Rossi, E., Chamberlain, B., Frasca, F., Eynard, D., Monti, F., Bronstein, M.M., 2020. Temporal graph networks for deep learning on dynamic graphs. *ICML 2020 Workshop on Graph Representation Learning* ArXiv:2006.10637.
- Rudin, C., 2019. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence* 1, 206–215.
- Sankar, A., Wu, Y., Gou, L., Zhang, W., Yang, H., 2020. DySAT: Deep neural representation learning on dynamic graphs via self-attention networks, in: *Proceedings of the 13th International Conference on Web Search and Data Mining (WSDM)*, pp. 519–527. doi:[10.1145/3336191.3371845](https://doi.org/10.1145/3336191.3371845). arXiv:1812.09430.
- Sargeant, H., 2026. Mind the gap: Securing algorithmic explainability for credit decisions beyond the uk gdpr. *Computer Law & Security Review* 60, 106247.
- Selbst, A.D., Powles, J., 2017. Meaningful information and the right to explanation. *International Data Privacy Law* 7, 233–242. doi:[10.1093/idpl/ix022](https://doi.org/10.1093/idpl/ix022).
- Shi, F., Cao, Y., Shang, Y., Zhou, Y., Zhou, C., Wu, J., 2022. H2-FDetector: A GNN-based fraud detector with homophilic and heterophilic connections, in: *Proceedings of the ACM Web Conference 2022 (WWW)*, pp. 1486–1494. doi:[10.1145/3485447.3512195](https://doi.org/10.1145/3485447.3512195).
- Simonyan, K., Vedaldi, A., Zisserman, A., 2014. Deep inside convolutional networks: Visualising image classification models and saliency maps, in: *International Conference on Learning Representations (ICLR), Workshop Track*.
- Sundararajan, M., Taly, A., Yan, Q., 2017. Axiomatic attribution for deep networks, in: *International Conference on Machine Learning (ICML)*, pp. 3319–3328.

- Tan, J., Geng, S., Fu, Z., Ge, Y., Xu, S., Li, Y., Zhang, Y., 2022. Learning and evaluating graph neural network explanations based on counterfactual and factual reasoning, in: Proceedings of the ACM Web Conference 2022 (WWW), pp. 1018–1027. doi:[10.1145/3485447.3512024](https://doi.org/10.1145/3485447.3512024). arXiv:2202.08816.
- Tang, J., 2025. Application of graphsage in financial fraud detection within dynamic social networks, in: 2025 International Conference on Electrical Drives, Power Electronics & Engineering (EDPEE), IEEE. pp. 989–993.
- Tang, J., Hua, F., Gao, Z., Zhao, P., Li, J., 2023. GADBench: Revisiting and benchmarking supervised graph anomaly detection, in: Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track.
- Tang, J., Li, J., Gao, Z., Li, J., 2022. Rethinking graph neural networks for anomaly detection, in: International Conference on Machine Learning (ICML), pp. 21076–21089.
- Teng, H.W., Härdle, W.K., Osterrieder, J., Pele, D.T., Baals, L.J., Papavasiliou, V., Bolesta, K., Kabašinskas, A., Filipovska, O., Thomaidis, N.S., et al., 2026. Digital assets: risks, regulations, mitigation. Financial Innovation 12, 65.
- U.S. Congress, 1974. Equal credit opportunity act. 15 U.S.C. § 1691 et seq., implemented by Regulation B (12 CFR Part 1002).
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., Bengio, Y., 2018. Graph attention networks, in: 6th International Conference on Learning Representations (ICLR). URL: <https://openreview.net/forum?id=rJXMpikCZ>, doi:[10.17863/CAM.48429](https://doi.org/10.17863/CAM.48429). arXiv:1710.10903.
- Wachter, S., Mittelstadt, B., Floridi, L., 2017. Why a right to explanation of automated decision-making does not exist in the general data protection regulation. International Data Privacy Law 7, 76–99. doi:[10.1093/idpl/ix005](https://doi.org/10.1093/idpl/ix005).
- Wang, X., Lyu, D., Li, M., Xia, Y., Yang, Q., Wang, X., Wang, X., Cui, P., Yang, Y., Sun, B., Guo, Z., 2021. APAN: Asynchronous propagation attention network for real-time temporal graph embedding, in: Proceedings

- of the 2021 International Conference on Management of Data (SIGMOD), pp. 2628–2638.
- Weber, M., Domeniconi, G., Chen, J., Weidele, D.K.I., Bellei, C., Robinson, T., Leiserson, C.E., 2019. Anti-money laundering in bitcoin: Experimenting with graph convolutional networks for financial forensics. arXiv preprint arXiv:1908.02591 .
- Xu, D., Ruan, C., Korpeoglu, E., Kumar, S., Achan, K., 2020. Inductive representation learning on temporal graphs, in: International Conference on Learning Representations (ICLR).
- Ying, Z., Bourgeois, D., You, J., Zitnik, M., Leskovec, J., 2019. GNNExplainer: Generating explanations for graph neural networks, in: Advances in Neural Information Processing Systems 32 (NeurIPS), pp. 9240–9251. ArXiv:1903.03894.
- You, J., Du, T., Leskovec, J., 2022. ROLAND: Graph learning framework for dynamic graphs, in: Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD), pp. 2358–2366. doi:[10.1145/3534678.3539300](https://doi.org/10.1145/3534678.3539300). arXiv:2208.07239.
- Yuan, H., Tang, J., Hu, X., Ji, S., 2020. XGNN: Towards model-level explanations of graph neural networks, in: Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), pp. 430–438. doi:[10.1145/3394486.3403085](https://doi.org/10.1145/3394486.3403085). arXiv:2006.02587.
- Yuan, H., Yu, H., Gui, S., Ji, S., 2023. Explainability in graph neural networks: A taxonomic survey. IEEE Transactions on Pattern Analysis and Machine Intelligence 45, 5782–5799. doi:[10.1109/TPAMI.2022.3204236](https://doi.org/10.1109/TPAMI.2022.3204236). arXiv:2012.15445.
- Yuan, H., Yu, H., Wang, J., Li, K., Ji, S., 2021. On explainability of graph neural networks via subgraph explorations, in: Proceedings of the 38th International Conference on Machine Learning (ICML), PMLR. pp. 12241–12252. URL: <http://proceedings.mlr.press/v139/yuan21c.html>.
- Zhang, G., Wu, J., Yang, J., Beheshti, A., Xue, S., Zhou, C., Sheng, Q.Z., 2021. FRAUDRE: Fraud detection dual-resistant to graph inconsis-

tency and imbalance, in: IEEE International Conference on Data Mining (ICDM), pp. 867–876.